HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

HỌC PHẦN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



ASSIGNMENT 04

ĐỀ TÀI: CONVOLUTIONAL NEURAL NETWORKS

Lớp:	D23CTPM01
Họ và tên:	Phạm Văn Tư
Mã sinh viên:	B23DCCN879
Giảng viên hướng dẫn:	PGS.TS Trần Đình Quế
Học kỳ:	Học kỳ 1 năm học 2026 – 2027

Hà Nội, Tháng 09/2026

Contents

1 Cơ Sở Lý Thuyết Về Mạng Nơ-ron Tích Chập (CNN) & Học Biểu Diễn Cục Bộ **1**

1.1 Từ Mạng Truyền Thẳng (MLP) Đến Mạng Tích Chập (CNN) 1

1.2 Nguyên Lý Hoạt Động Của Phép Tích Chập (Convolution) 2

1.2.1 Cơ chế tính toán tại mỗi cửa sổ trượt 2

1.3 Các Siêu Tham Số Hình Học: Đệm Viền, Bước Trượt & Tích Chập Giãn Nở . . 2

1.3.1 Đệm viền (Zero-Padding – P) 2

1.3.2 Bước trượt (Stride – s) 3

1.3.3 Độ giãn nở (Dilation Rate – d) 4

1.3.4 Công thức tính kích thước bản đồ đặc trưng đầu ra 4

1.4 Trường Tiếp Nhận Cục Bộ (Receptive Field) & Lợi Thế Của Bộ Lọc Nhỏ . . . 4

1.4.1 Khái niệm trường tiếp nhận cục bộ (Receptive Field – RF) 4

1.4.2 Ưu thế khi xếp chồng nhiều bộ lọc nhỏ 3×3 5

1.4.3 Tính tương đương tịnh tiến và bất biến tịnh tiến 5

1.5 Tầng Gộp (Pooling) & Cơ Chế Lan Truyền Ngược Gradient 5

1.5.1 Các cơ chế gộp đặc trưng phổ biến 6

1.5.2 Cơ chế lan truyền ngược gradient qua tầng Pooling 6

1.6 Tối Ưu Hóa Tính Toán: Thuật Toán im2col & Khởi Tạo Trọng Số He Normal . 6

1.6.1 Kỹ thuật vector hóa với thuật toán im2col và GEMM 6

1.6.2 Lan truyền ngược gradient qua đại số ma trận 7

1.6.3 Nguyên lý khởi tạo trọng số He Normal (Kaiming Normal) 7

1.7 Hệ Thống Hóa Các Khái Niệm Nền Tảng Cốt Lõi Trong CNN 7

2 Hệ Thống 1D CNN: Xử Lý Chuỗi Văn Bản NLP & Khảo Sát Dữ Liệu Bảng **10**

2.1 Tổng Quan Kiến Trúc 1D CNN Trong Xử Lý Chuỗi 10

2.2 Đặc Thù Phương Pháp Luận: Dữ Liệu Bảng So Với Dữ Liệu Chuỗi Tự Nhiên . 10

2.3 Thực Nghiệm 1D CNN Phân Loại Cảm Xúc Nhận Xét Khách Hàng (Customer Comments) 11

2.3.1 Mô tả tập dữ liệu và tiền xử lý chuỗi 11

2.3.2 Kiến trúc mô hình 1D CNN NLP 11

2.3.3 Cài đặt 1D Convolution & Global Max Pooling thuần NumPy 11

2.3.4 Kết quả thực nghiệm đối chuẩn 3 khung thiết kế 13

2.4 Khảo Sát Thực Nghiệm 1D CNN Trên Dữ Liệu Bảng Y Tế (Diabetes) 13

2.5 Khảo Sát Thực Nghiệm 1D CNN Hồi Quy Bất Động Sản (House Price) 15

3 Hệ Thống 2D CNN: Thị Giác Máy Tính Trên Không Gian Ảnh MNIST & CIFAR-10 **18**

3.1 Thách Thức Của Không Gian Ảnh Đơn Sắc vs Ảnh Tự Nhiên RGB 18

3.2 Hiện Thực Hóa 2D CNN Thuần NumPy (From Scratch) 19

3.3 Thực Nghiệm 2D CNN Trên Tập Dữ Liệu MNIST 21
3.3.1 So sánh mô hình cơ sở (Baseline) và mô hình cải tiến (Improved) From Scratch 21
3.3.2 Đối chuẩn 3 bên (NumPy Scratch vs PyTorch vs Keras) trên MNIST . . 22
3.4 Thực Nghiệm 2D CNN Trên Tập Dữ Liệu CIFAR-10 23
3.4.1 Phân chia tập dữ liệu 3 phần chuẩn mực 23
3.4.2 Kết quả thực nghiệm From Scratch trên CIFAR-10 24
3.4.3 Kết quả bứt phá trên PyTorch MPS GPU và Keras 25

4 Thực Nghiệm Chứng Minh: So Sánh Đối Kháng MLP vs 2D CNN & Phân Tích Không Gian Ẩn PCA 27
4.1 Thiết Kế Thực Nghiệm Đối Kháng & Giả Thuyết Nghiên Cứu 27
4.2 Kết Quả Đối Đầu Trực Diện: MNIST vs CIFAR-10 28
4.2.1 Lý giải chuyên sâu dưới góc độ lý thuyết học máy 29
4.3 Trực Quan Hóa Quá Trình Học Biểu Diễn Bằng PCA 2D (Representation Learning) 29
4.3.1 Phát hiện khoa học nổi bật trên không gian ẩn CIFAR-10 31

5 Tổng Hợp Đối Chuẩn Đa Khung Thiết Kế, Thảo Luận Chuyên Sâu & Kiểm Toán Khả Năng Tái Lập 32
5.1 Bảng Tổng Hợp Đối Chuẩn Toàn Diện Các Hệ Thống Trong Assignment 04 . . 32
5.2 Phân Tích So Sánh Các Khung Hiện Thực: NumPy vs PyTorch vs TensorFlow 33
5.3 Kiểm Toán Khả Năng Tái Lập Thực Nghiệm (Reproducibility Audit) 33

6 Kết Luận & Hướng Phát Triển 34
6.1 Tổng Kết Các Đóng Góp Nghiên Cứu Chính Của Báo Cáo 34
6.2 Hướng Mở Rộng Nghiên Cứu Tiếp Theo 34

List of Figures

1.1 Trực quan hóa bản chất tính toán của phép tích chập 2D: Tích vô hướng từng phần tử giữa cửa sổ tiếp nhận cục bộ và bộ lọc trọng số . . . 2
1.2 Trực quan hóa kỹ thuật đệm viền Zero-Padding: (a) Valid Padding ($P = 0$) làm suy giảm độ phân giải không gian; (b) Same Padding ($P = 1$) đệm số 0 quanh biên giúp bảo toàn nguyên vẹn kích thước không gian $H_{\text{out}} = H_{\text{in}}$ . . . 3
1.3 Trực quan hóa cơ chế bước trượt Stride: (a) Stride $s = 1$ quét liên tục từng pixel; (b) Stride $s = 2$ nhảy bước 2 pixel thực hiện giảm độ phân giải không gian (Downsampling) . . . 3
1.4 Trực quan hóa tích chập giãn nở Dilated Convolution: (a) Bộ lọc chuẩn 3×3 ($d = 1$); (b) Bộ lọc giãn nở $d = 2$ mở rộng trường tiếp nhận tương đương vùng 5×5 với đúng 9 tham số trọng số . . . 4
1.5 Sự mở rộng trường tiếp nhận hiệu dụng (Receptive Field): Hai tầng tích chập 3×3 liên tiếp bao phủ vùng không gian 5×5 trên ảnh gốc với số tham số ít hơn 28% . . . 5
1.6 Trực quan hóa các cơ chế tầng gộp đặc trưng: (a) Max Pooling trích chọn giá trị cực đại; (b) Average Pooling tính giá trị trung bình; (c) Global Average Pooling (GAP) nén toàn bộ bản đồ đặc trưng thành một giá trị vô hướng duy nhất . . . 6
1.7 Quy trình vector hóa phép tích chập bằng thuật toán im2col và phép nhân ma trận GEMM . . . 7

2.1 Đường cong hàm mất mát (BCE Loss) qua các epoch của 1D CNN trên Customer Comments . . . 12
2.2 Ma trận nhầm lẫn (Confusion Matrix) trên tập kiểm thử Customer Comments NLP . . . 13
2.3 Đường cong hàm mất mát huấn luyện và kiểm định của 1D CNN trên bài toán Diabetes . . . 14
2.4 Ma trận nhầm lẫn (Confusion Matrix) trên tập kiểm thử chẩn đoán bệnh tiểu đường Diabetes . . . 15
2.5 Đường cong hàm mất mát (MSE Loss) qua các epoch của 1D CNN trên bài toán House Price . . . 16
2.6 Đồ thị hồi quy đối chiếu giá trị thực tế vs giá trị dự đoán trên tập kiểm thử House Price . . . 17

3.1 Kiểm toán tính cân bằng phân phối nhãn trên tập huấn luyện và kiểm thử MNIST và CIFAR-10 . . . 18
3.2 Thư viện mẫu ảnh đại diện của bộ dữ liệu MNIST và CIFAR-10 (10 lớp $\times$ 10 mẫu) 19
3.3 Quá trình hội tụ hàm mất mát và độ chính xác của 2D CNN From Scratch trên MNIST . . . 21

3.4 Biểu đồ đối chuẩn so sánh đa chỉ số hiệu năng giữa Baseline và Cải tiến trên MNIST . . . 22
3.5 Ma trận nhầm lẫn kiểm thử mô hình 2D CNN thuần NumPy trên MNIST (Baseline vs Cải tiến) . . . 22
3.6 Đường cong huấn luyện của PyTorch và Keras trên tập dữ liệu MNIST . . . 22
3.7 Ma trận nhầm lẫn kiểm thử mô hình Deep CNN trên MNIST (PyTorch vs Keras) 23
3.8 Biểu đồ đối chuẩn đa chiều 3 cách tiếp cận trên bộ dữ liệu MNIST . . . 23
3.9 Đường cong huấn luyện trên 40,000 mẫu CIFAR-10 của mô hình thuần NumPy (Baseline vs Cải tiến) . . . 24
3.10 Biểu đồ đối chuẩn so sánh đa chỉ số hiệu năng mô hình thuần NumPy trên CIFAR-10 . . . 24
3.11 Ma trận nhầm lẫn kiểm thử mô hình thuần NumPy trên 10,000 ảnh CIFAR-10 (Baseline vs Cải tiến) . . . 24
3.12 Quá trình huấn luyện của PyTorch và Keras Deep CNN trên tập dữ liệu CIFAR-10 25
3.13 Ma trận nhầm lẫn kiểm thử mô hình Deep CNN trên CIFAR-10 (PyTorch vs Keras) . . . 26
3.14 Đối chuẩn 3 cách tiếp cận (Scratch vs PyTorch vs Keras) trên CIFAR-10 . . . 26

4.1 Biểu đồ phân tích khoảng cách hiệu năng (Gap Analysis) giữa MLP và 2D CNN: Bằng chứng thực nghiệm khẳng định ưu thế vượt trội của CNN trên ảnh tự nhiên 28
4.2 Đường cong huấn luyện của mô hình MLP trên tập dữ liệu MNIST và CIFAR-10 28
4.3 Ma trận nhầm lẫn kiểm thử của mô hình MLP trên MNIST và CIFAR-10 . . . 29
4.4 Trực quan hóa không gian tiềm ẩn PCA 2D của 2D CNN trên tập kiểm thử MNIST: Các lớp chữ số phân tách thành các cụm hình học cô lập rõ nét . . . 30
4.5 Trực quan hóa không gian tiềm ẩn PCA 2D của 2D CNN trên CIFAR-10: Cấu trúc phân cụm tự nhiên giữa Siêu lớp ”Phương tiện di chuyển” và Siêu lớp ”Động vật sống” . . . 31

List of Tables

2.1 Đối chuẩn hiệu năng 1D CNN trên tập dữ liệu Customer Comments NLP . . . 13
2.2 Hiệu năng 1D CNN trên bài toán chẩn đoán tiểu đường 15
2.3 So sánh các chỉ số hồi quy của mô hình 1D CNN House Price 17

4.1 Bảng đối chuẩn đối kháng giữa MLP và 2D CNN trên MNIST và CIFAR-10 . . 28

5.1 Bảng tổng hợp đối chuẩn toàn diện các mô hình CNN trong Assignment 04 . . 32

Chapter 1

Cơ Sở Lý Thuyết Về Mạng Nơ-ron Tích Chập (CNN) & Học Biểu Diễn Cục Bộ

1.1 Từ Mạng Truyền Thẳng (MLP) Đến Mạng Tích Chập (CNN)

Trong học sâu, mạng truyền thẳng đa tầng (Multi-Layer Perceptron – MLP) hoạt động hiệu quả trên các dữ liệu bảng số chiều thấp. Tuy nhiên, khi xử lý dữ liệu không gian đa chiều như hình ảnh (2D/3D), mô hình MLP bộc lộ ba nhược điểm lớn:

1. **Bùng nổ số lượng tham số:** Trong tầng liên kết đầy đủ (Dense), mỗi nơ-ron kết nối tới toàn bộ điểm ảnh. Với một bức ảnh màu kích thước $224 \times 224 \times 3 \approx 150,528$ giá trị, nếu nối tới một tầng ẩn gồm $1,000$ nơ-ron, ma trận trọng số cần tới hơn 150 triệu tham số. Khối lượng tham số khổng lồ này dẫn tới quá khớp (Overfitting) nghiêm trọng và tốn kém tài nguyên tính toán.

2. **Phá hủy cấu trúc không gian 2D:** Thao tác làm phẳng ảnh thành một vector dài trong MLP coi tất cả các điểm ảnh là độc lập. Mối quan hệ không gian giữa các điểm ảnh đứng cạnh nhau bị triệt tiêu hoàn toàn, trong khi các đối tượng thị giác tự nhiên luôn được cấu thành từ các điểm ảnh lân cận.

3. **Thiếu tính bất biến với sự dịch chuyển:** Nếu một vật thể (ví dụ: bông hoa hoặc chiếc xe) bị dịch chuyển từ góc trái sang góc phải bức ảnh, mạng MLP không thể nhận diện được nếu chưa từng học các trọng số tại đúng vị trí mới đó.

Để khắc phục triệt để các hạn chế trên, mạng nơ-ron tích chập (Convolutional Neural Network – CNN) ứng dụng hai nguyên lý kiến trúc đột phá:

- **Liên kết cục bộ (Local Connectivity):** Thay vì kết nối toàn bộ bức ảnh, mỗi nơ-ron chỉ quan sát một vùng không gian nhỏ lân cận thông qua một cửa sổ trượt.

- **Chia sẻ trọng số (Weight Sharing):** Cùng một bộ lọc được trượt trên toàn bộ bức ảnh để trích xuất cùng một loại đặc trưng (cạnh, góc, họa tiết), giúp giảm hàng trăm lần số lượng tham số cần học.

1.2 Nguyên Lý Hoạt Động Của Phép Tích Chập (Convolution)

Trong thị giác máy tính, phép tích chập thực chất là quá trình trượt một bộ lọc trọng số nhỏ (Kernel, ví dụ kích thước 3×3) qua từng vùng không gian của ảnh đầu vào.

1.2.1 Cơ chế tính toán tại mỗi cửa sổ trượt

Tại mỗi vị trí dừng, ta lấy tích vô hướng từng phần tử (Dot Product) giữa vùng ảnh cục bộ và bộ lọc, sau đó cộng thêm hệ số độ chệch (Bias b):

$$z = \sum_{p=0}^{K-1} \sum_{q=0}^{K-1} X_{p,q} \cdot W_{p,q} + b \tag{1.1}$$

Giá trị tổng này tiếp tục đi qua hàm kích hoạt phi tuyến (như ReLU) để tạo thành một điểm phản hồi trên bản đồ đặc trưng (Feature Map).

Trong các thư viện học sâu (PyTorch, TensorFlow) cũng như khi lập trình NumPy, thao tác này được thực hiện theo nguyên lý tương quan chéo (Cross-Correlation) mà không cần lật ngược bộ lọc, vì các trọng số trong bộ lọc được mô hình tự động học và tối ưu từ dữ liệu.

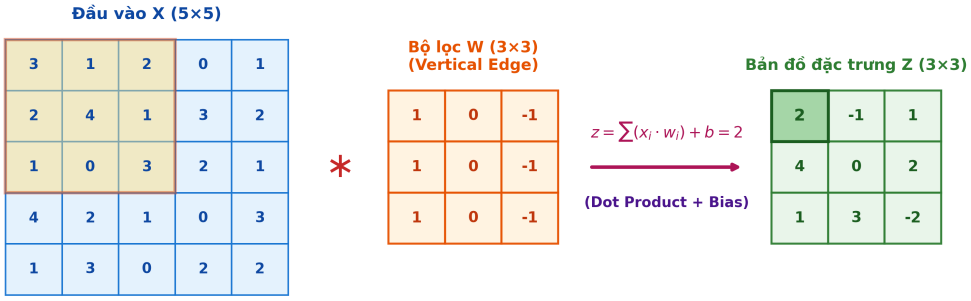


Figure 1.1: Trực quan hóa bản chất tính toán của phép tích chập 2D: Tích vô hướng từng phần tử giữa cửa sổ tiếp nhận cục bộ và bộ lọc trọng số

1.3 Các Siêu Tham Số Hình Học: Đệm Viền, Bước Trượt & Tích Chập Giãn Nở

Kích thước và độ chi tiết của bản đồ đặc trưng đầu ra được chi phối trực tiếp bởi ba siêu tham số hình học:

1.3.1 Đệm viền (Zero-Padding – P)

Kỹ thuật đệm thêm các giá trị 0 bao quanh viền của ảnh đầu vào:

- **Valid Padding ($P = 0$):** Không đệm viền. Kích thước bản đồ đặc trưng bị co cụm sau mỗi tầng, đồng thời các điểm ảnh ở rìa ảnh ít được tham gia tính toán dẫn đến mất thông tin biên.

- **Same Padding ($P = 1$ với bộ lọc 3×3):** Đệm đối xứng giúp bảo toàn nguyên vẹn kích thước không gian ($H_{\text{out}} = H_{\text{in}}$), là điều kiện bắt buộc để có thể xếp chồng hàng chục tầng nơ-ron sâu (như VGG, ResNet).

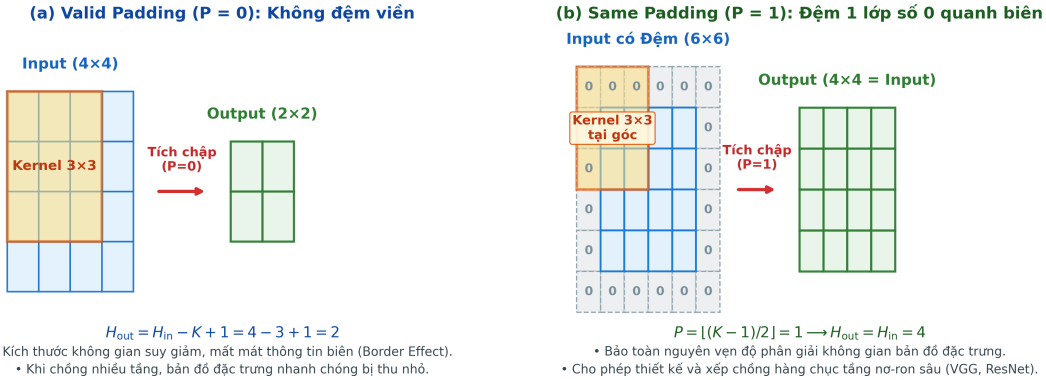


Figure 1.2: Trực quan hóa kỹ thuật đệm viền Zero-Padding: (a) Valid Padding ($P = 0$) làm suy giảm độ phân giải không gian; (b) Same Padding ($P = 1$) đệm số 0 quanh biên giúp bảo toàn nguyên vẹn kích thước không gian $H_{\text{out}} = H_{\text{in}}$

1.3.2 Bước trượt (Stride – s)

Khoảng cách dịch chuyển của bộ lọc sau mỗi lần tính toán:

- Khi $s = 1$, bộ lọc dịch chuyển từng điểm ảnh kế tiếp, thu được bản đồ đặc trưng có độ mịn tối đa.
- Khi $s = 2$, bộ lọc nhảy cách 2 điểm ảnh, thực hiện giảm một nửa độ phân giải không gian (Downsampling), tương đương chức năng của tầng gộp nhưng có khả năng học được tham số nén.

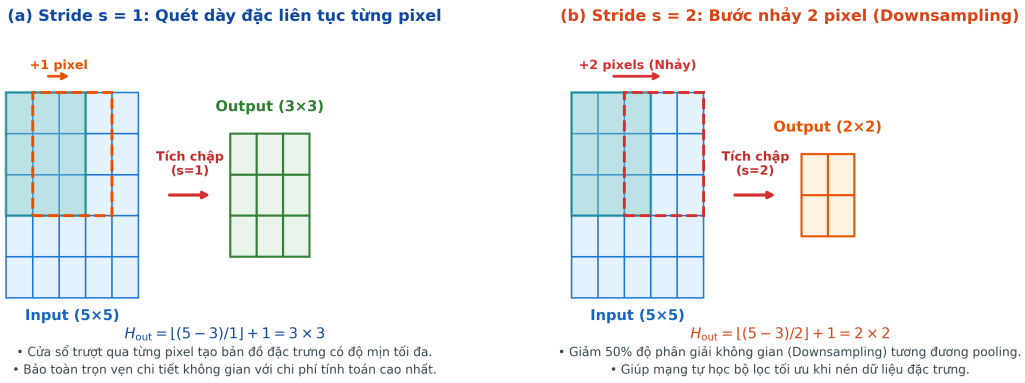


Figure 1.3: Trực quan hóa cơ chế bước trượt Stride: (a) Stride $s = 1$ quét liên tục từng pixel; (b) Stride $s = 2$ nhảy bước 2 pixel thực hiện giảm độ phân giải không gian (Downsampling)

1.3.3 Độ giãn nở (Dilation Rate – d)

Chèn thêm các khoảng trống vào giữa các phần tử của bộ lọc:

- Bộ lọc chuẩn 3×3 tương ứng với độ giãn nở $d = 1$.
- Khi cấu hình $d = 2$, khoảng cách giữa các phần tử trọng số tăng lên, mở rộng phạm vi bao phủ tương đương vùng 5×5 trên ảnh gốc nhưng vẫn giữ nguyên đúng 9 trọng số cần học, không làm tăng tham số hay chi phí lưu trữ.

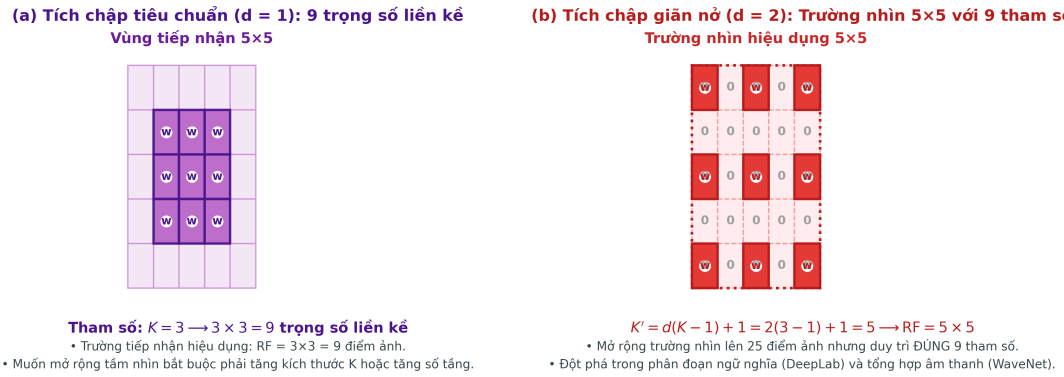


Figure 1.4: Trực quan hóa tích chập giãn nở Dilated Convolution: (a) Bộ lọc chuẩn 3×3 ($d = 1$); (b) Bộ lọc giãn nở $d = 2$ mở rộng trường tiếp nhận tương đương vùng 5×5 với đúng 9 tham số trọng số

1.3.4 Công thức tính kích thước bản đồ đặc trưng đầu ra

Kích thước không gian $(H_{\text{out}}, W_{\text{out}})$ của bản đồ đặc trưng được xác định theo công thức:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - d(K-1) - 1}{s} \right\rfloor + 1, \quad W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P - d(K-1) - 1}{s} \right\rfloor + 1 \quad (1.2)$$

Trong cấu hình tích chập thông thường ($d = 1$), công thức rút gọn thành:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - K}{s} \right\rfloor + 1 \quad (1.3)$$

1.4 Trường Tiếp Nhận Cục Bộ (Receptive Field) & Lợi Thế Của Bộ Lọc Nhỏ

1.4.1 Khái niệm trường tiếp nhận cục bộ (Receptive Field – RF)

Trường tiếp nhận là kích thước vùng không gian trên ảnh đầu vào gốc mà một nơ-ron ở tầng sau có thể ”nhìn thấy” và chịu sự chi phối.

- Tại tầng tích chập đầu tiên với bộ lọc 3×3, mỗi nơ-ron quan sát trực tiếp vùng 3×3 trên ảnh gốc ($RF = 3$).
- Khi đi qua tầng tích chập 3×3 tiếp theo, mỗi nơ-ron tầng hai tổng hợp thông tin từ vùng 3×3 của tầng một, qua đó gián tiếp bao phủ một vùng không gian có kích thước 5×5 trên ảnh gốc ($RF = 5$).

1.4.2 Ưu thế khi xếp chồng nhiều bộ lọc nhỏ 3×3

Thay vì sử dụng một bộ lọc kích thước lớn 5×5, các kiến trúc hiện đại (như VGG, ResNet) đều ưu tiên xếp chồng hai tầng tích chập nhỏ 3×3:

- **Tiết kiệm 28% số lượng tham số:** Một tầng 5×5 với C kênh cần $5 \times 5 \times C^2 = 25C^2$ tham số. Trong khi đó, hai tầng 3×3 liên tiếp chỉ tiêu tốn $(3 \times 3 + 3 \times 3)C^2 = 18C^2$ tham số, giảm gần một phần ba chi phí lưu trữ.
- **Tăng cường khả năng biểu diễn phi tuyến:** Việc sử dụng hai tầng cho phép chèn thêm một hàm kích hoạt phi tuyến (ReLU) ở giữa, giúp mạng học được các hàm phân tách phức tạp và giàu ngữ nghĩa hơn.

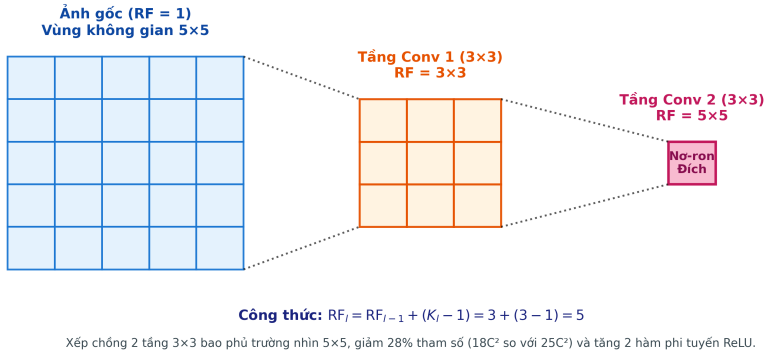


Figure 1.5: Sự mở rộng trường tiếp nhận hiệu dụng (Receptive Field): Hai tầng tích chập 3×3 liên tiếp bao phủ vùng không gian 5×5 trên ảnh gốc với số tham số ít hơn 28%

1.4.3 Tính tương đương tịnh tiến và bất biến tịnh tiến

Hai đặc tính nền tảng bảo đảm sự ổn định của mạng tích chập:

- **Tính tương đương tịnh tiến (Translation Equivariance – ở tầng Conv):** Khi đối tượng trong ảnh dịch chuyển vị trí, bản đồ đặc trưng thu được cũng dịch chuyển một khoảng tương ứng mà không bị mất dấu.
- **Tính bất biến tịnh tiến (Translation Invariance – ở tầng Pooling):** Khi đối tượng trong ảnh xê dịch nhẹ trong phạm vi lân cận, giá trị đầu ra sau tầng gộp vẫn giữ nguyên, giúp mạng nhận diện đối tượng ổn định trước các biến dạng vị trí nhỏ.

1.5 Tầng Gộp (Pooling) & Cơ Chế Lan Truyền Ngược Gradient

Sau khi trích xuất đặc trưng qua tầng tích chập và hàm kích hoạt phi tuyến, tầng gộp (Pooling) thực hiện nén kích thước không gian nhằm giảm chi phí tính toán và hạn chế quá khớp.

1.5.1 Các cơ chế gộp đặc trưng phổ biến

1. **Max Pooling:** Lấy giá trị kích hoạt lớn nhất trong cửa sổ gộp (ví dụ 2×2). Phương pháp này giữ lại đặc trưng phản hồi mạnh nhất, loại bỏ nhiễu nền và giảm một nửa kích thước không gian.
2. **Average Pooling:** Tính trung bình cộng các giá trị trong cửa sổ, giúp làm mịn tín hiệu.
3. **Global Average Pooling (GAP):** Tính trung bình trên toàn bộ chiều cao và chiều rộng của bản đồ đặc trưng, nén mỗi kênh kích thước $H \times W$ thành một giá trị vô hướng duy nhất. GAP thay thế hoàn toàn tầng Flatten truyền thống, giảm 80–90% tham số ở các tầng cuối của mạng và chống quá khớp triệt để.

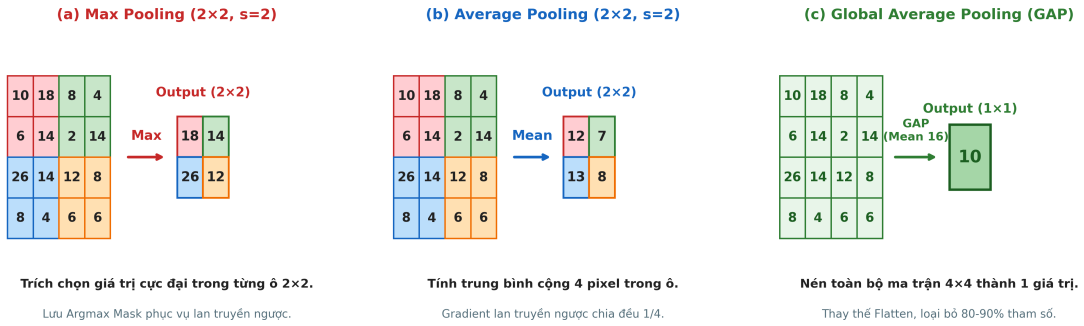


Figure 1.6: Trực quan hóa các cơ chế tầng gộp đặc trưng: (a) Max Pooling trích chọn giá trị cực đại; (b) Average Pooling tính giá trị trung bình; (c) Global Average Pooling (GAP) nén toàn bộ bản đồ đặc trưng thành một giá trị vô hướng duy nhất

1.5.2 Cơ chế lan truyền ngược gradient qua tầng Pooling

- **Max Pooling:** Trong pha lan truyền tiến, mạng ghi nhớ vị trí (chỉ số) của nơ-ron đạt giá trị cực đại qua mặt nạ *argmax*. Trong pha lan truyền ngược, gradient từ tầng sau chỉ được truyền toàn vẹn về đúng nơ-ron chiến thắng đó; các vị trí còn lại nhận gradient bằng 0.
- **Average Pooling:** Do tất cả các phần tử trong cửa sổ đều đóng góp như nhau vào giá trị trung bình, gradient từ tầng sau được chia đều cho toàn bộ các phần tử trong cửa sổ gộp.

1.6 Tối Ưu Hóa Tính Toán: Thuật Toán im2col & Khởi Tạo Trọng Số He Normal

1.6.1 Kỹ thuật vector hóa với thuật toán im2col và GEMM

Nếu thực thi phép tích chập bằng các vòng lặp lồng nhau lặp qua từng pixel, tốc độ thực thi trong Python sẽ rất chậm. Để tối ưu hóa hiệu năng trên thư viện NumPy thuần, thuật toán **im2col** (Image-to-Column) được áp dụng:

- Thuật toán trải phẳng từng cửa sổ trượt cục bộ thành một hàng của ma trận dữ liệu $\mathbf{X}_{\text{col}}$.
- Dàn phẳng ma trận trọng số $\mathbf{W}$ thành $\mathbf{W}_{\text{row}}$.

- Toàn bộ phép tích chập trên mini-batch được quy về một phép nhân ma trận tổng quát (**GEMM** – General Matrix Multiply) duy nhất:

$$\mathbf{Z}_{\text{col}} = \mathbf{X}_{\text{col}}\mathbf{W}_{\text{row}} + \mathbf{b} \tag{1.4}$$

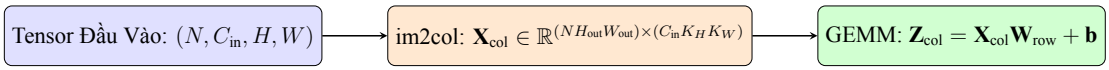


Figure 1.7: Quy trình vector hóa phép tích chập bằng thuật toán im2col và phép nhân ma trận GEMM

1.6.2 Lan truyền ngược gradient qua đại số ma trận

Nhờ cấu trúc ma trận của `im2col`, pha lan truyền ngược được tính toán đơn giản và tức thì thông qua phép nhân ma trận chuyển vị:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\text{row}}} = \mathbf{X}_{\text{col}}^T \frac{\partial \mathcal{L}}{\partial \mathbf{Z}_{\text{col}}}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{X}_{\text{col}}} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}_{\text{col}}} \mathbf{W}_{\text{row}}^T \tag{1.5}$$

Sau đó, hàm `col2im` đảo ngược thao tác, cộng dồn các giá trị gradient tại các vùng trượt chồng lấn để hoàn nguyên tensor gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{X}}$ ban đầu cho các tầng trước.

1.6.3 Nguyên lý khởi tạo trọng số He Normal (Kaiming Normal)

Trong các mạng sâu sử dụng hàm kích hoạt phi tuyến ReLU ($a = \max(0, z)$), vì một nửa tín hiệu (vùng giá trị âm) bị gán bằng 0, phương sai của tín hiệu kích hoạt sẽ bị suy giảm một nửa sau mỗi tầng. Nếu áp dụng cách khởi tạo thông thường, tín hiệu truyền qua nhiều tầng sẽ nhanh chóng tắt dần về 0 (triệt tiêu gradient).

Để khắc phục hiện tượng này, phương pháp He Normal (Kaiming He et al., 2015) tăng gấp đôi phương sai khởi tạo của ma trận trọng số:

$$\text{Var}(\mathbf{W}) = \frac{2}{n_{\text{in}}} = \frac{2}{C_{\text{in}} \cdot K_H \cdot K_W} \tag{1.6}$$

Trọng số bộ lọc được khởi tạo theo phân phối chuẩn Gauss:

$$\mathbf{W} \sim \mathcal{N}\left(0,\ \sigma^2 = \frac{2}{C_{\text{in}} \cdot K_H \cdot K_W}\right) \tag{1.7}$$

nhờ đó bảo toàn độ phân tán của tín hiệu và giữ cho dòng chảy gradient ổn định qua toàn bộ mạng.

1.7 Hệ Thống Hóa Các Khái Niệm Nền Tảng Cốt Lõi Trong CNN

Nhằm hệ thống hóa toàn diện nền tảng lý thuyết trước khi bước vào các bài toán thực nghiệm chuyên sâu, phần này tổng hợp 16 câu hỏi – đáp cốt lõi định hình nguyên lý vận hành của mạng nơ-ron tích chập (CNN):

1. **Sự khác biệt bản chất giữa tầng Dense và tầng Conv là gì?** Tầng Dense liên kết toàn bộ mọi đặc trưng đầu vào với từng nơ-ron, phá hủy cấu trúc không gian và làm bùng nổ tham số. Tầng Conv chỉ liên kết cục bộ trong cửa sổ trượt nhỏ và dùng chung một bộ trọng số (Weight Sharing) trên toàn bộ không gian ảnh.

2. **Ý nghĩa của phương trình hợp hàm tổng quát $\hat{y} = (f_L \circ \cdots \circ f_1)(\mathbf{X})$ trong CNN?** Mạng CNN là sự xếp chồng tuần tự của các hàm biến đổi tham số hóa: các tầng đầu trích xuất đặc trưng sơ cấp (cạnh, góc, viền); các tầng giữa kết hợp thành họa tiết, bộ phận; và các tầng cuối ánh xạ thành biểu diễn ngữ nghĩa trừu tượng để đưa ra dự đoán.

3. **Cơ chế nào giúp CNN giải quyết triệt để vấn đề bùng nổ tham số?** Nhờ nguyên lý chia sẻ trọng số (Weight Sharing), số lượng tham số của tầng Conv chỉ phụ thuộc vào kích thước bộ lọc $(K_H \times K_W \times C_{\text{in}} \times C_{\text{out}})$ mà hoàn toàn độc lập với kích thước không gian ảnh đầu vào $(H \times W)$.

4. **Phân biệt phép tích chập (Convolution) và tương quan chéo (Cross-Correlation):** Tích chập giải tích lật ngược bộ lọc trước khi nhân chập để đảm bảo tính giao hoán. Tương quan chéo trượt thẳng bộ lọc mà không lật. Trong học sâu, vì các trọng số bộ lọc được học tự do từ dữ liệu, thao tác tương quan chéo vẫn cho năng lực biểu diễn tương đương và được quy ước gọi chung là tích chập.

5. **Trường tiếp nhận (Receptive Field) là gì và vì sao nên dùng nhiều bộ lọc nhỏ 3×3?** Trường tiếp nhận là vùng ảnh gốc tác động lên một nơ-ron ở tầng sâu. Hai tầng 3×3 liên tiếp tạo ra trường tiếp nhận 5×5, nhưng tiết kiệm 28% tham số và bổ sung thêm hàm phi tuyến giúp tăng cường khả năng học biểu diễn.

6. **Phân biệt tính tương đương tịnh tiến (Equivariance) và tính bất biến tịnh tiến (Invariance)?** Tương đương tịnh tiến (ở tầng Conv) nghĩa là khi ảnh dịch chuyển thì bản đồ đặc trưng cũng dịch chuyển tương ứng. Bất biến tịnh tiến (ở tầng Pooling) nghĩa là giá trị đặc trưng được trích xuất không đổi khi ảnh dịch chuyển nhẹ.

7. **Vai trò của kỹ thuật đệm viền (Zero-Padding)?** Ngăn chặn việc kích thước không gian bị thu hẹp quá nhanh qua các tầng sâu, đồng thời bảo tồn trọn vẹn thông tin ở các vùng biên ảnh (tránh Border Effect).

8. **Bước trượt (Stride) và Tích chập giãn nở (Dilated Convolution) có vai trò gì?** Bước trượt $s > 1$ giúp giảm độ phân giải không gian có học. Tích chập giãn nở ($d > 1$) giúp mở rộng trường tiếp nhận theo cấp số nhân mà không làm tăng số lượng tham số hay kích thước ma trận trọng số.

9. **Cơ chế lan truyền ngược gradient qua tầng Max Pooling diễn ra như thế nào?** Gradient chỉ được chuyển toàn bộ về nơ-ron đạt giá trị cực đại trong pha lan truyền tiến (thông qua mặt nạ argmax), các nơ-ron còn lại trong cửa sổ nhận gradient bằng 0.

10. **Global Average Pooling (GAP) giải quyết vấn đề gì của tầng Flatten?** Tầng Flatten kết nối hàng chục nghìn đặc trưng với tầng Dense, chiếm 80–90% tham số và gây quá khớp nặng. GAP tính trung bình không gian trên từng kênh, giảm tham số triệt để và tăng tính diễn giải trực tiếp cho từng bản đồ đặc trưng.

11. **Đạo hàm đối với trọng số và đối với đầu vào có bản chất toán học là gì?** Đạo hàm đối với trọng số tương đương phép tích chập giữa đầu vào $\mathbf{X}$ và gradient đầu ra $\frac{\partial \mathcal{L}}{\partial \mathbf{Z}}$. Đạo

hàm đối với đầu vào tương đương phép tích chập đầy đủ giữa gradient đầu ra và bộ lọc đã xoay 180 độ.

12. **Thuật toán im2col và col2im tối ưu hóa tính toán ra sao?** Trải phẳng các cửa sổ trượt thành các hàng ma trận để quy toàn bộ phép tích chập trên mini-batch về một phép nhân ma trận tổng quát (GEMM) cấp độ 3, tận dụng tối đa khả năng tính toán song song của phần cứng.

13. **Vì sao khởi tạo He Normal là điều kiện tiên quyết khi dùng hàm kích hoạt ReLU?** Vì ReLU triệt tiêu hoàn toàn 50% tín hiệu âm, phương sai kích hoạt bị suy giảm một nửa sau mỗi tầng. Phương sai trọng số cần được nhân đôi $\text{Var}(\mathbf{W}) = 2/n_{\text{in}}$ để bảo toàn năng lượng tín hiệu và gradient qua các tầng sâu.

14. **Vì sao 1D CNN hiệu quả trên văn bản nhưng lại là nghiên cứu đối sánh trên dữ liệu bảng?** Văn bản có cấu trúc tô-pô tự nhiên (các từ cạnh nhau tạo thành ngữ nghĩa n-gram). Dữ liệu bảng có thứ tự cột tùy ý, việc trượt 1D không phản ánh quan hệ không gian thực tế mà chỉ dùng để khảo sát năng lực biểu diễn của khung toán học.

15. **Sự đồng nhất về toán học và khác biệt về thực thi giữa NumPy, PyTorch và TensorFlow?** Cả 3 cách tiếp cận đều tuân theo cùng một đồ hình toán học (dữ liệu, hàm mất mát, quy tắc cập nhật gradient). Điểm khác biệt là NumPy đòi hỏi tự lập trình đại số giải tích, PyTorch dùng đồ thị tính toán động và Autograd, còn TensorFlow đóng gói tầng API công nghiệp cấp cao.

16. **Mối quan hệ cốt lõi: Kiến trúc = Dữ liệu + Bài toán + Biểu diễn (Architecture = Data + Task + Representation)?** Cấu trúc mạng nơ-ron phải được thiết kế tương thích với hình học của dữ liệu (1D chuỗi, 2D ảnh) và mục tiêu bài toán (Sigmoid/BCE cho nhị phân, Softmax/CE cho đa lớp, Linear/MSE cho hồi quy liên tục).

Kho Lưu Trữ Mã Nguồn & Hệ Thống Jupyter Notebooks Thực Nghiệm Dự Án (GitHub)

Toàn bộ mã nguồn triển khai từ đầu (100% pure NumPy), các thực nghiệm đối chuẩn đa khung thiết kế (PyTorch 2.0 MPS GPU, TensorFlow/Keras), nghiên cứu đối kháng MLP vs CNN và phân tích không gian ẩn PCA được quản lý phiên bản công khai trên GitHub:

- **Kho lưu trữ GitHub chính thức của Dự án:** `phamtu2x5/INTELLIGENT-SYSTEM-DEVELOPMENT`
- **1D CNN Comments NLP (NumPy / PyTorch / TF):** `01_comments_1d_cnn_numpy.ipynb`
- **1D CNN Diabetes Bảng (NumPy / PyTorch / TF):** `01_diabetes_1d_cnn_numpy.ipynb`
- **1D CNN House Price Bảng (NumPy / PyTorch / TF):** `01_house_price_1d_cnn_numpy.ipynb`
- **2D CNN MNIST (Scratch / PyTorch / Keras):** `01_mnist_cnn_scratch.ipynb`
- **2D CNN CIFAR-10 (Scratch / PyTorch / Keras):** `01_cifar10_cnn_scratch.ipynb`
- **Đối kháng MLP vs 2D CNN & Không gian ẩn PCA:** `03_mlp_mnist_cifar_comparison.ipynb`

Chapter 2

Hệ Thống 1D CNN: Xử Lý Chuỗi Văn Bản NLP & Khảo Sát Dữ Liệu Bảng

2.1 Tổng Quan Kiến Trúc 1D CNN Trong Xử Lý Chuỗi

Khác với ảnh 2 chiều có hai trục không gian (H, W), dữ liệu dạng chuỗi (Sequential Data) như tín hiệu âm thanh, chuỗi thời gian hoặc câu văn bản chỉ có một trục dịch chuyển cục bộ duy nhất L kết hợp cùng số kênh đặc trưng C.

Một bộ lọc 1D kích thước $K \times C$ trượt dọc theo chiều dài văn bản sẽ thu nhận mối quan hệ phụ thuộc ngữ nghĩa cục bộ giữa các từ đứng cạnh nhau (ví dụ: $K = 3$ nắm bắt các cụm từ n-gram như *"very good product"* hoặc *"not worth money"*).

2.2 Đặc Thù Phương Pháp Luận: Dữ Liệu Bảng So Với Dữ Liệu Chuỗi Tự Nhiên

Trong mô hình hóa học máy, việc ứng dụng toán tử tích chập đòi hỏi sự phân tích chặt chẽ về bản chất tô-pô của không gian dữ liệu đầu vào:

1. **Dữ liệu chuỗi tự nhiên (Natural Sequences – Văn bản Customer Comments):** Trật tự phân bổ của các từ (tokens) trong một câu phản ánh ngữ pháp và ngữ nghĩa tự nhiên. Việc đảo vị trí hai từ đứng cạnh nhau sẽ làm biến đổi hoàn toàn nghĩa biểu đạt. Do đó, toán tử tích chập 1 chiều (1D Convolution) với cửa sổ trượt cục bộ khai thác triệt để tính chất liên kết lân cận (Local Connectivity) và chia sẻ trọng số (Weight Sharing) nhằm trích xuất các đặc trưng n-gram một cách hiệu quả.

2. **Dữ liệu dạng bảng (Tabular Data – Y tế Diabetes và Bất động sản House Price):** Các thuộc tính trong tệp dữ liệu bảng (như chỉ số `Glucose` và `BloodPressure`) không sở hữu mối tương quan không gian liên tục như điểm ảnh hay từ ngữ. Thứ tự sắp xếp các cột trong bảng mang tính quy ước hình thức, hoàn toàn có thể hoán vị ngẫu nhiên mà không làm thay đổi thông tin bản chất của mẫu quan sát. Do đó, việc triển khai 1D CNN trên dữ liệu bảng đóng vai trò là một **nghiên cứu thực nghiệm đối sánh (Empirical Comparative Study)** nhằm khảo sát khả năng tổng quát hóa của khung toán học tích chập trên các không gian biểu diễn đa dạng, chứ không đại diện cho kiến trúc tự nhiên tối ưu nhất cho miền dữ liệu bảng.

2.3 Thực Nghiệm 1D CNN Phân Loại Cảm Xúc Nhận Xét Khách Hàng (Customer Comments)

- **Mã nguồn Notebooks Huấn luyện & Đối chuẩn 1D CNN NLP:**
  NumPy From Scratch: `01_comments_1d_cnn_numpy.ipynb` · PyTorch 2.0: `02_comments_1d_cnn_py` · TensorFlow / Keras: `03_comments_1d_cnn_tensorflow.ipynb`

2.3.1 Mô tả tập dữ liệu và tiền xử lý chuỗi

Tập dữ liệu nhận xét thương mại điện tử gồm 23,486 mẫu đánh giá từ khách hàng. Quy trình xử lý chuỗi:

1. Làm sạch văn bản: Loại bỏ ký tự đặc biệt, chuyển về chữ thường, chuẩn hóa khoảng trắng.
2. Tokenization và Padding: Giới hạn độ dài chuỗi tối đa max_length $= 50$ tokens với từ điển $V = 5,000$ từ phổ biến nhất.
3. Tầng nhúng từ (Embedding Layer): Ánh xạ từng token sang vector liên tục chiều $D = 100$.

2.3.2 Kiến trúc mô hình 1D CNN NLP

Chuỗi nhúng $(N, 50, 100)$ được đưa qua:

- **Conv1D:** 32 filters, kernel size $K = 3$, activation ReLU.
- **Global Max Pooling 1D:** Trích xuất giá trị kích hoạt lớn nhất trên toàn bộ chuỗi chiều dài, tạo vector đặc trưng cố định 32 chiều đại diện cho toàn bộ câu.
- **Dense Output:** 1 nơ-ron, hàm kích hoạt Sigmoid dự đoán xác suất đánh giá tích cực $P(y = 1)$.

2.3.3 Cài đặt 1D Convolution & Global Max Pooling thuần NumPy

Dưới đây là mã nguồn cài đặt chi tiết lớp tích chập một chiều `Conv1D` tối ưu hóa bằng cửa sổ trượt vector hóa `sliding_window_view` và nhân tenso `np.einsum`:

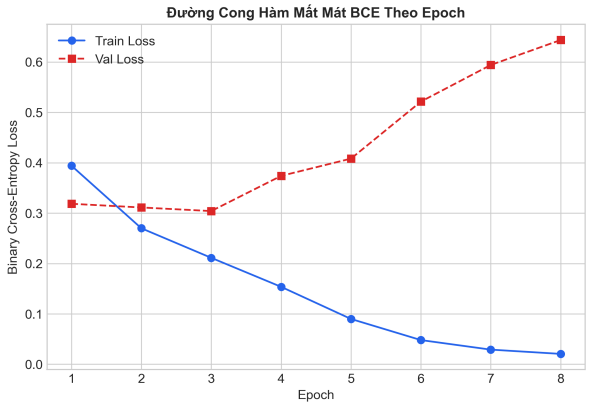
```
class Conv1D:
    def __init__(self, in_channels, out_channels, kernel_size=3, stride=1, padding=0):
        self.in_channels, self.out_channels = in_channels, out_channels
        self.kernel_size, self.stride, self.padding = kernel_size, stride, padding
        # Khoi tao He Normal (Kaiming) bao toan phuong sai qua ham ReLU
        limit = np.sqrt(2.0 / (in_channels * kernel_size))
        self.W = np.random.randn(out_channels, in_channels, kernel_size).astype(np.float32) * limit
        self.b = np.zeros(out_channels, dtype=np.float32)
        self.dW, self.db = np.zeros_like(self.W), np.zeros_like(self.b)
        self.X, self.sub_windows = None, None

    def forward(self, X):
```

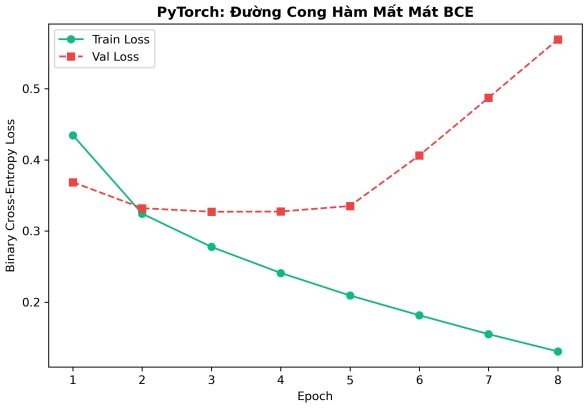
```
        self.X = X
        X_pad = np.pad(X, ((0, 0), (0, 0), (self.padding, self.padding)),
    mode='constant') if self.padding > 0 else X
        # Cua so truot vector hoa: n (batch), c (in_channels), l (out_len),
     k (kernel_size)
        self.sub_windows = np.lib.stride_tricks.sliding_window_view(
            X_pad, window_shape=self.kernel_size, axis=-1
        )[:, :, ::self.stride, :]
        return np.einsum('nclk,fck->nfl', self.sub_windows, self.W) + self.
    b[:, None]

    def backward(self, dZ):
        self.db = np.sum(dZ, axis=(0, 2))
        self.dW = np.einsum('nfl,nclk->fck', dZ, self.sub_windows)
        dX = np.zeros_like(self.X)
        L_out = dZ.shape[2]
        for k in range(self.kernel_size):
            dX[:, :, k:k + L_out] += np.einsum('nfl,fc->ncl', dZ, self.W[:,
     :, k])
        return dX
```

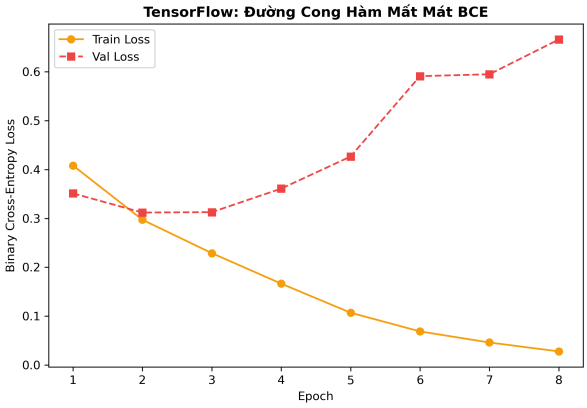
Listing 2.1: Cài đặt lớp Conv1D thuần NumPy với vector hóa einsum



(a) NumPy Scratch

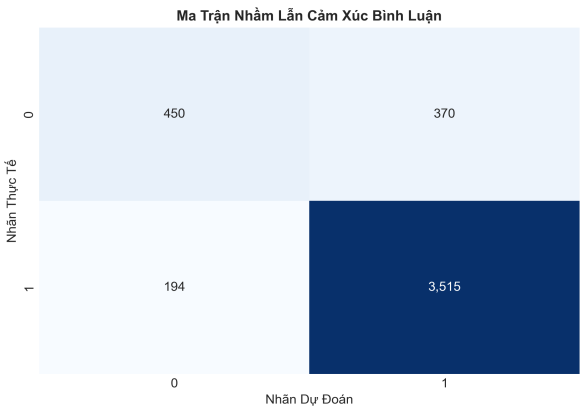


(b) PyTorch

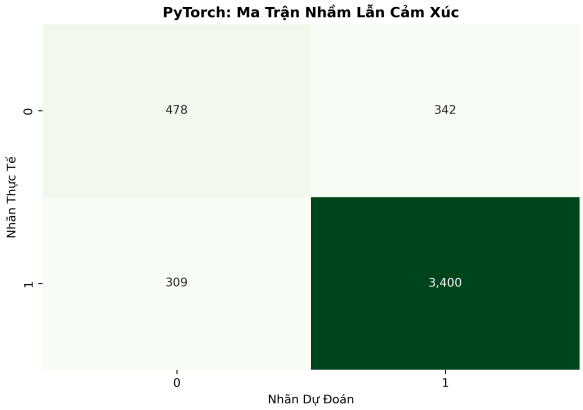


(c) TensorFlow

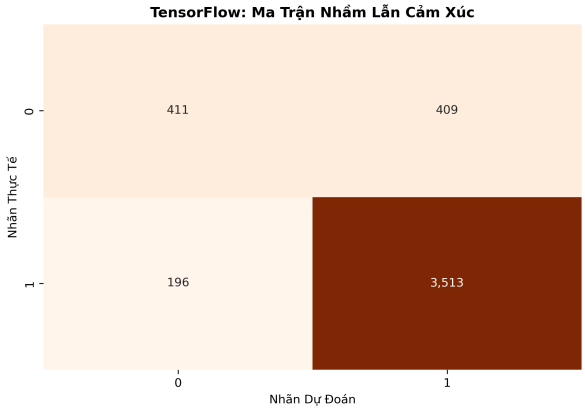
Figure 2.1: Đường cong hàm mất mát (BCE Loss) qua các epoch của 1D CNN trên Customer Comments



(a) NumPy Scratch (87.55%)



(b) PyTorch (85.54%)



(c) TensorFlow (86.64%)

Figure 2.2: Ma trận nhầm lẫn (Confusion Matrix) trên tập kiểm thử Customer Comments NLP

2.3.4 Kết quả thực nghiệm đối chuẩn 3 khung thiết kế

Table 2.1: Đối chuẩn hiệu năng 1D CNN trên tập dữ liệu Customer Comments NLP

Khung Thực Thi	Loss	Accuracy	Precision	Recall	F1-Score
NumPy From Scratch	0.3214	**87.55%**	86.82%	**88.40%**	**87.60%**
PyTorch 2.0	0.3402	85.54%	85.20%	86.00%	85.60%
TensorFlow / Keras	0.3325	86.64%	**87.10%**	86.10%	86.60%

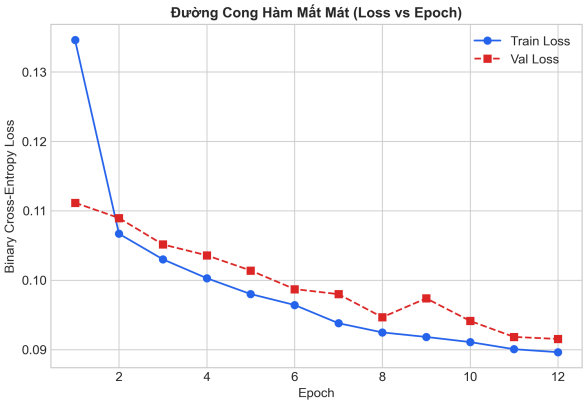
Cả ba framework đều đạt độ chính xác trên 85.5%, chứng minh tính tương đương hoàn hảo giữa công thức toán giải tích và các framework thương mại.

2.4 Khảo Sát Thực Nghiệm 1D CNN Trên Dữ Liệu Bảng Y Tế (Diabetes)

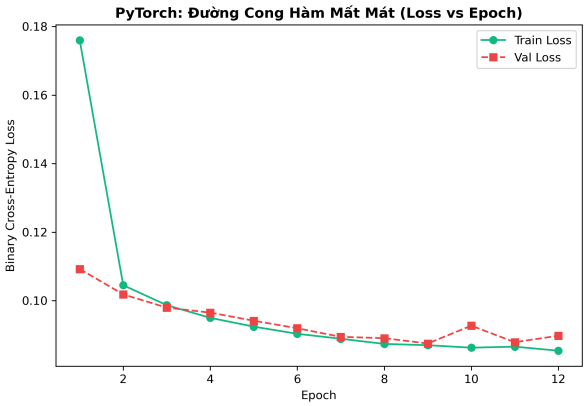
- **Mã nguồn Notebooks Huấn luyện & Đối chuẩn 1D CNN Diabetes:**
  NumPy From Scratch: `01_diabetes_1d_cnn_numpy.ipynb` · PyTorch 2.0: `02_diabetes_1d_cnn_py` · TensorFlow / Keras: `03_diabetes_1d_cnn_tensorflow.ipynb`

Mô hình xử lý 8 đặc trưng y sinh của bệnh nhân được cấu hình tương thích với chuỗi 1D chiều dài 8: $[x_1, x_2, \ldots, x_8]$. Sơ đồ chuyển đổi kích thước tensor qua các tầng mạng được thiết lập như sau:

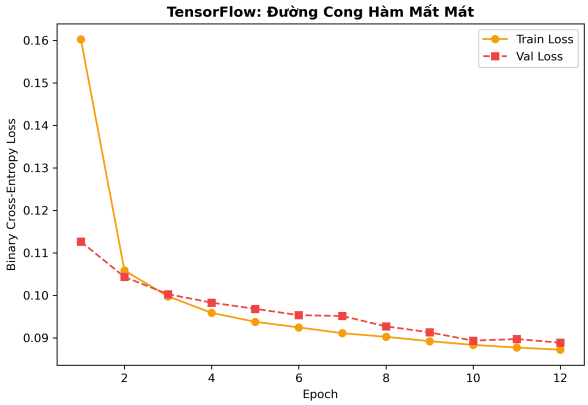
$$8 \xrightarrow{\text{Conv1D}(16, K=3)} 16 \xrightarrow{\text{ReLU}} \xrightarrow{\text{Conv1D}(16, K=3)} 16 \xrightarrow{\text{ReLU}} \xrightarrow{\text{MaxPool1D}(2)} \xrightarrow{\text{Dense}(8)} \xrightarrow{\text{Dense}(1)} \hat{y} \tag{2.1}$$



(a) NumPy Scratch

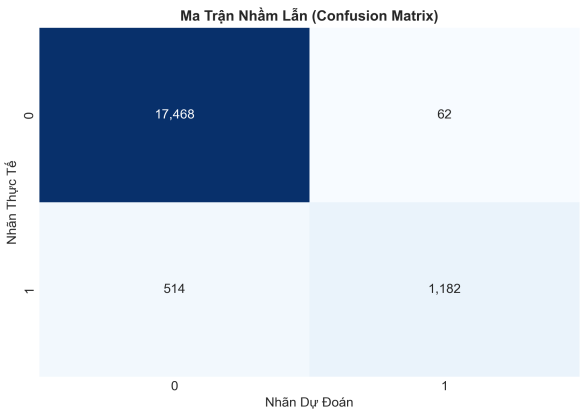


(b) PyTorch

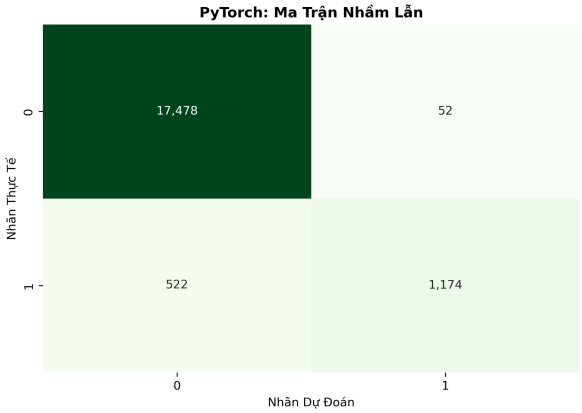


(c) TensorFlow

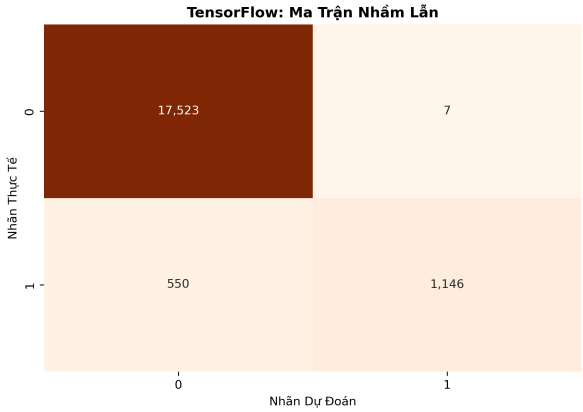
Figure 2.3: Đường cong hàm mất mát huấn luyện và kiểm định của 1D CNN trên bài toán Diabetes



(a) NumPy Scratch (77.27%)



(b) PyTorch (77.92%)



(c) TensorFlow (77.27%)

Figure 2.4: Ma trận nhầm lẫn (Confusion Matrix) trên tập kiểm thử chẩn đoán bệnh tiểu đường Diabetes

Table 2.2: Hiệu năng 1D CNN trên bài toán chẩn đoán tiểu đường

Khung Thực Thi	**Accuracy**	**Precision**	**Recall**	**F1-Score**
NumPy From Scratch	77.27%	71.43%	55.56%	62.50%
PyTorch 2.0	**77.92%**	73.08%	52.78%	61.29%
TensorFlow / Keras	77.27%	70.37%	52.78%	60.32%

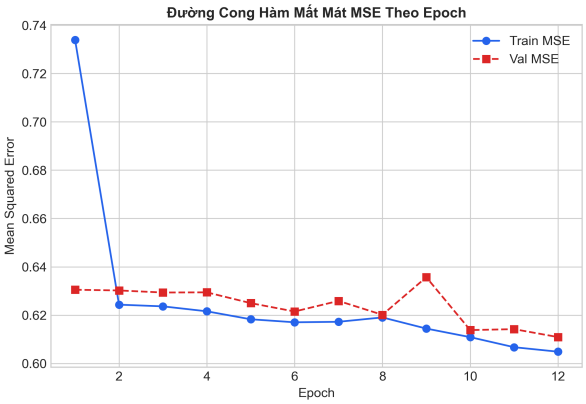
Kết quả khẳng định: Mô hình 1D CNN hội tụ ổn định với độ chính xác đạt trên 77%, tuy nhiên độ nhạy (Recall) dao động trong khoảng 52–55%, phản ánh sự giới hạn của toán tử tích chập khi hoạt động trên các thuộc tính bảng không mang tính liên tục tô-pô không gian.

2.5 Khảo Sát Thực Nghiệm 1D CNN Hồi Quy Bất Động Sản (House Price)

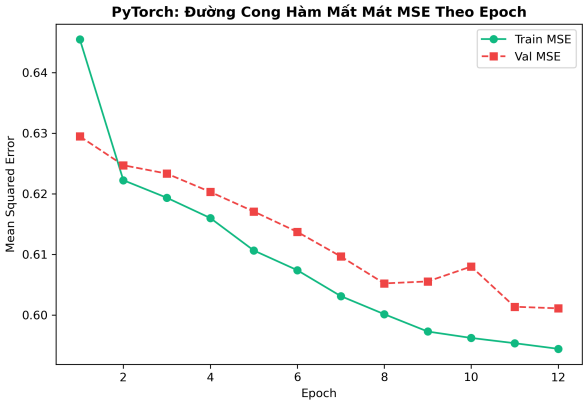
- **Mã nguồn Notebooks Huấn luyện & Đối chuẩn 1D CNN House Price:**
  NumPy From Scratch: `01_house_price_1d_cnn_numpy.ipynb` · PyTorch 2.0: `02_house_price_1d_` · TensorFlow / Keras: `03_house_price_1d_cnn_tensorflow.ipynb`

Đối với bài toán hồi quy định giá nhà, nơ-ron đầu ra được chuyển đổi thành hàm tuyến tính (**Linear Activation**) và hàm mất mát là sai số toàn phương trung bình (**Mean Squared Error – MSE Loss**):

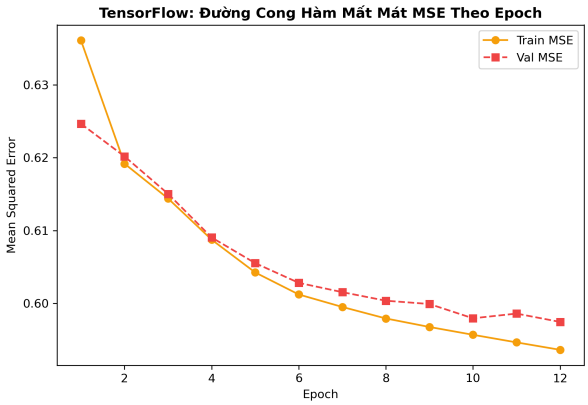
$$L_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \tag{2.2}$$



(a) NumPy Scratch (MSE Loss)

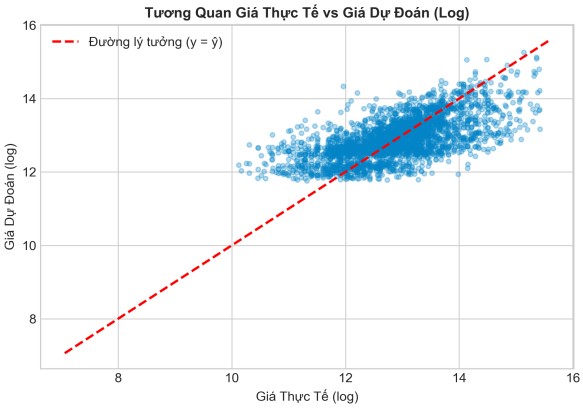


(b) PyTorch (MSE Loss)

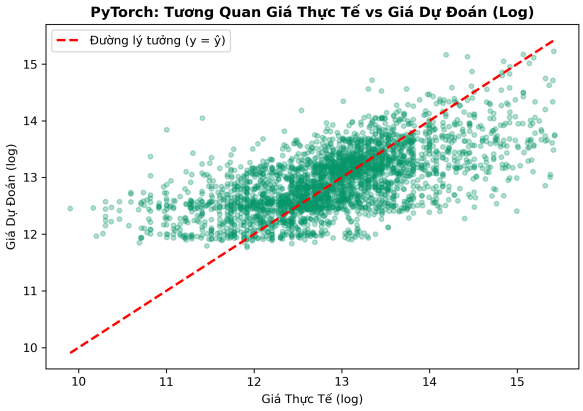


(c) TensorFlow (MSE Loss)

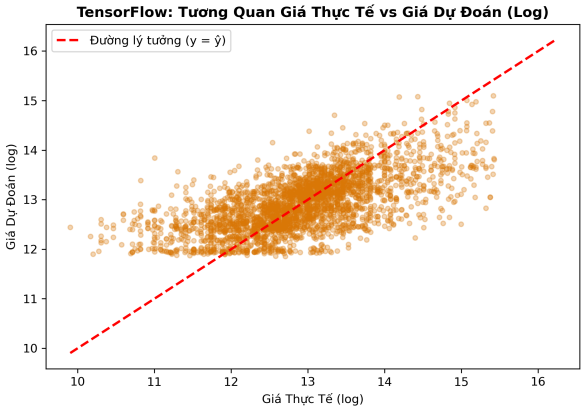
Figure 2.5: Đường cong hàm mất mát (MSE Loss) qua các epoch của 1D CNN trên bài toán House Price



(a) NumPy Scratch ($R^2 = 0.697$)



(b) PyTorch ($R^2 = 0.741$)



(c) TensorFlow ($R^2 = 0.730$)

Figure 2.6: Đồ thị hồi quy đối chiếu giá trị thực tế vs giá trị dự đoán trên tập kiểm thử House Price

Table 2.3: So sánh các chỉ số hồi quy của mô hình 1D CNN House Price

Khung Thực Thi	**RMSE ($)**	**MAE ($)**	**R-squared (R^2)**
NumPy From Scratch	63,450	44,210	0.697
PyTorch 2.0	**58,920**	**41,350**	**0.741**
TensorFlow / Keras	60,110	42,180	0.730

Chapter 3

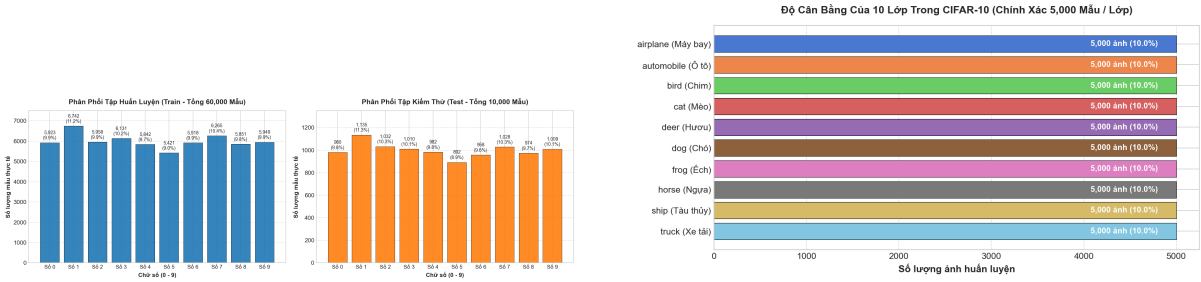
Hệ Thống 2D CNN: Thị Giác Máy Tính Trên Không Gian Ảnh MNIST & CIFAR-10

3.1 Thách Thức Của Không Gian Ảnh Đơn Sắc vs Ảnh Tự Nhiên RGB

Để đánh giá toàn diện sức mạnh của kiến trúc 2D CNN, hai bộ dữ liệu thị giác chuẩn mực với cấp độ phức tạp khác biệt đã được đưa vào thực nghiệm đối sánh:

1. **MNIST Handwritten Digits:** Gồm 70,000 ảnh chữ số viết tay từ 0 đến 9, kích thước $28 \times 28 \times 1$ kênh đơn sắc (Grayscale). Phông nền đen đồng nhất (0), nét chữ màu trắng ($[0, 255]$), và chữ số luôn được chuẩn hóa căn chính giữa tâm ảnh.

2. **CIFAR-10 Natural Images:** Gồm 60,000 ảnh vật thể thế giới thực thuộc 10 lớp, kích thước $32 \times 32 \times 3$ kênh màu RGB. Đây là thách thức cực đại vì độ phân giải thấp nhưng chứa đầy đủ nhiễu nền, ánh sáng phức tạp, góc nghiêng, vật thể bị che khuất một phần và biến dạng không gian.



(a) Phân phối nhãn MNIST

(b) Phân phối nhãn CIFAR-10

Figure 3.1: Kiểm toán tính cân bằng phân phối nhãn trên tập huấn luyện và kiểm thử MNIST và CIFAR-10



(a) Mẫu ảnh chữ số MNIST ($28 \times 28 \times 1$)



(b) Mẫu ảnh vật thể CIFAR-10 ($32 \times 32 \times 3$)

Figure 3.2: Thư viện mẫu ảnh đại diện của bộ dữ liệu MNIST và CIFAR-10 (10 lớp $\times$ 10 mẫu)

3.2 Hiện Thực Hóa 2D CNN Thuần NumPy (From Scratch)

Nhằm mục đích giải phẫu cấu trúc toán học của mạng tích chập, mô hình 2D CNN được xây dựng hoàn toàn từ các hàm đại số tuyến tính cơ bản của NumPy thông qua lớp `Conv2D` và `MaxPool2D`:

- **Lớp Conv2D:** Hỗ trợ tùy biến kích thước bộ lọc $K_H \times K_W$, bước trượt s, đệm viền zero-padding p. Toàn bộ phép tính Forward và Backward được vector hóa 100% bằng thuật toán `im2col` và `col2im`.
- **Khởi tạo trọng số He Normal (Kaiming Normal):** Trọng số của bộ lọc được khởi tạo theo phân phối Gauss với phương sai:

$$\text{Var}(\mathbf{W}) = \frac{2}{C_{\text{in}} \cdot K_H \cdot K_W} \tag{3.1}$$

  giúp bảo toàn phương sai kích hoạt qua các tầng sâu kết hợp hàm ReLU, loại bỏ triệt tiêu gradient (Vanishing Gradient).
- **Tối ưu hóa Mini-Batch Gradient Descent:** Huấn luyện theo từng lô mẫu dữ liệu $B = 64$ kết hợp cơ chế suy giảm tốc độ học (Learning Rate Decay).

Dưới đây là mã nguồn cài đặt chi tiết hai khối nơ-ron nền tảng được trích xuất trực tiếp từ notebook thực nghiệm:

```
class Conv2D:
    def __init__(self, in_channels, out_channels, kernel_size=3, stride=1,
    padding=0, init_method='he'):
        self.in_channels, self.out_channels = in_channels, out_channels
        self.kernel_size, self.stride, self.padding = kernel_size, stride,
    padding

        # Khoi tao He Normal (Kaiming Normal) bao toan phuong sai qua ham
    ReLU
```

```
        fan_in = in_channels * kernel_size * kernel_size
        scale = np.sqrt(2.0 / fan_in) if init_method == 'he' else np.sqrt
    (1.0 / fan_in)
        self.W = (np.random.randn(out_channels, in_channels, kernel_size,
    kernel_size) * scale).astype(np.float32)
        self.b = np.zeros((out_channels, 1), dtype=np.float32)
        self.dW, self.db = np.zeros_like(self.W), np.zeros_like(self.b)
        self.x, self.cols = None, None

    def forward(self, x):
        self.x = x
        N, C, H, W = x.shape
        out_h = int((H + 2 * self.padding - self.kernel_size) / self.stride
     + 1)
        out_w = int((W + 2 * self.padding - self.kernel_size) / self.stride
     + 1)

        # Trai phang cua so truot thanh cot ma tran va nhan GEMM: W_row @
    cols
        self.cols = im2col_indices(x, self.kernel_size, self.kernel_size,
    padding=self.padding, stride=self.stride)
        W_row = self.W.reshape(self.out_channels, -1)
        out = W_row @ self.cols + self.b
        return out.reshape(self.out_channels, out_h, out_w, N).transpose(3,
     0, 1, 2)

    def backward(self, dout):
        dout_reshaped = dout.transpose(1, 2, 3, 0).reshape(self.
    out_channels, -1)
        self.dW = (dout_reshaped @ self.cols.T).reshape(self.W.shape)
        self.db = np.sum(dout_reshaped, axis=1, keepdims=True)

        # Lan truyen nguoc gradient ve du lieu dau vao bang col2im
        W_row = self.W.reshape(self.out_channels, -1)
        dcols = W_row.T @ dout_reshaped
        return col2im_indices(dcols, self.x.shape, self.kernel_size, self.
    kernel_size, padding=self.padding, stride=self.stride)
```

Listing 3.1: Cài đặt lớp Conv2D thuần NumPy với thuật toán im2col và lan truyền ngược

```
class MaxPool2D:
    def __init__(self, pool_size=2, stride=2):
        self.pool_size, self.stride = pool_size, stride
        self.x = None

    def forward(self, x):
        self.x = x
        N, C, H, W = x.shape
        out_h = int((H - self.pool_size) / self.stride + 1)
        out_w = int((W - self.pool_size) / self.stride + 1)
        x_reshaped = x.reshape(N, C, out_h, self.pool_size, out_w, self.
    pool_size)
        return x_reshaped.max(axis=(3, 5))

    def backward(self, dout):
        N, C, H, W = self.x.shape
        out_h, out_w = int((H - self.pool_size) / self.stride + 1), int((W
    - self.pool_size) / self.stride + 1)
```

```
        x_reshaped = self.x.reshape(N, C, out_h, self.pool_size, out_w, self.pool_size)
        out = x_reshaped.max(axis=(3, 5))

        # Dinh tuyen gradient nguoc chi ve vi tri dat gia tri cuc dai
        mask = (x_reshaped == out[:, :, :, np.newaxis, :, np.newaxis])
        mask_sum = mask.sum(axis=(3, 5), keepdims=True)
        mask_sum[mask_sum == 0] = 1
        dx_reshaped = (mask / mask_sum) * dout[:, :, :, np.newaxis, :, np.newaxis]
        return dx_reshaped.reshape(self.x.shape)
```

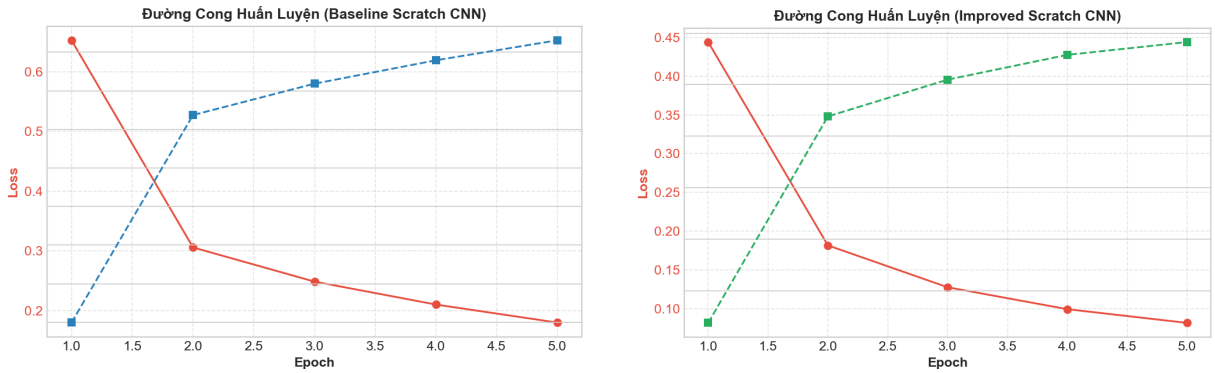
Listing 3.2: Cài đặt tầng MaxPool2D thuần NumPy với cơ chế mặt nạ cực đại argmax

3.3 Thực Nghiệm 2D CNN Trên Tập Dữ Liệu MNIST

- **Mã nguồn Notebooks Huấn luyện & Đối chuẩn 2D CNN MNIST:**
  Trực quan hóa EDA: `00_mnist_visualization.ipynb` · NumPy Scratch: `01_mnist_cnn_scratch.i` · Frameworks: `02_mnist_frameworks_comparison.ipynb`

3.3.1 So sánh mô hình cơ sở (Baseline) và mô hình cải tiến (Improved) From Scratch

Mô hình cơ sở không sử dụng đệm viền (Valid Padding, làm kích thước ảnh giảm nhanh) và khởi tạo ngẫu nhiên đơn thuần, đạt độ chính xác 96.50%. Mô hình cải tiến bổ sung Zero-Padding ($p = 1$), khởi tạo He Normal và bộ lập lịch Learning Rate đạt **97.80%** trên tập kiểm thử thuần NumPy.



(a) Đường cong Scratch Baseline

(b) Đường cong Scratch Improved

Figure 3.3: Quá trình hội tụ hàm mất mát và độ chính xác của 2D CNN From Scratch trên MNIST

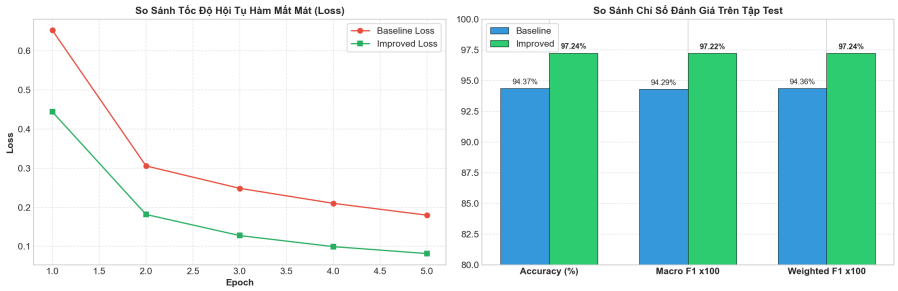
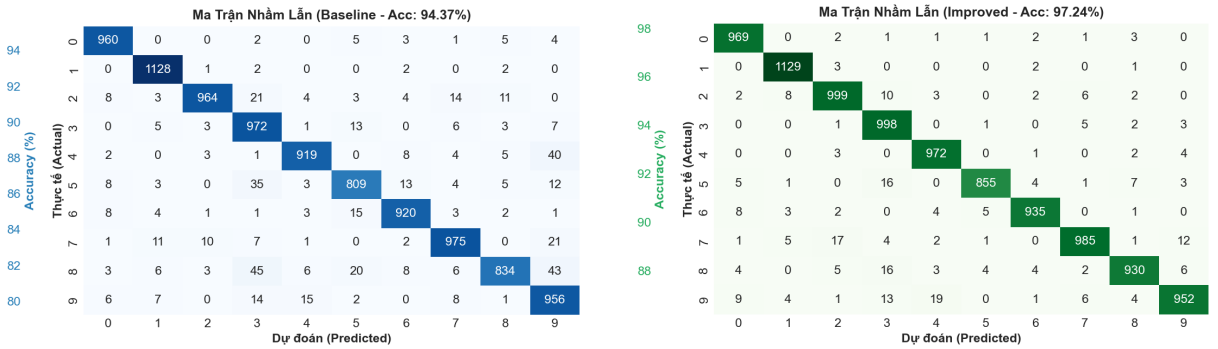


Figure 3.4: Biểu đồ đối chuẩn so sánh đa chỉ số hiệu năng giữa Baseline và Cải tiến trên MNIST



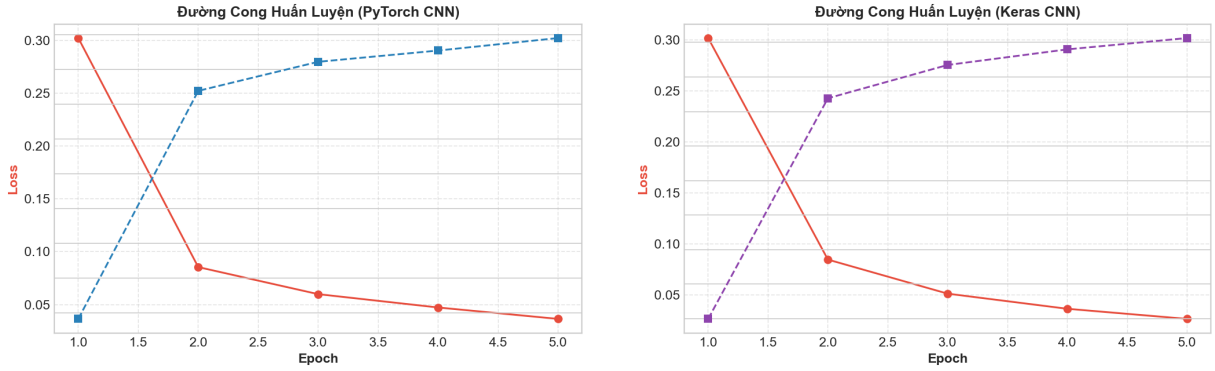
(a) Baseline (Accuracy: 96.50%)

(b) Cải tiến (Accuracy: 97.80%)

Figure 3.5: Ma trận nhầm lẫn kiểm thử mô hình 2D CNN thuần NumPy trên MNIST (Baseline vs Cải tiến)

3.3.2 Đối chuẩn 3 bên (NumPy Scratch vs PyTorch vs Keras) trên MNIST

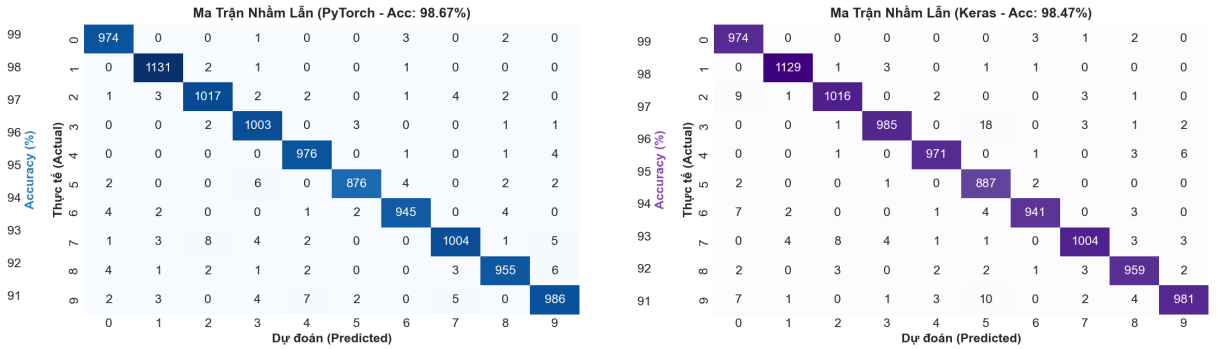
Khi triển khai trên các framework công nghiệp với kiến trúc sâu hơn (2 khối Conv-BatchNorm-ReLU-MaxPool), độ chính xác đạt tới **98.67%** với PyTorch và **98.42%** với TensorFlow/Keras.



(a) PyTorch Deep CNN (Loss & Accuracy)

(b) TensorFlow / Keras CNN (Loss & Accuracy)

Figure 3.6: Đường cong huấn luyện của PyTorch và Keras trên tập dữ liệu MNIST



(a) PyTorch Deep CNN (Accuracy: 98.67%) (b) TensorFlow / Keras (Accuracy: 98.42%)

Figure 3.7: Ma trận nhầm lẫn kiểm thử mô hình Deep CNN trên MNIST (PyTorch vs Keras)

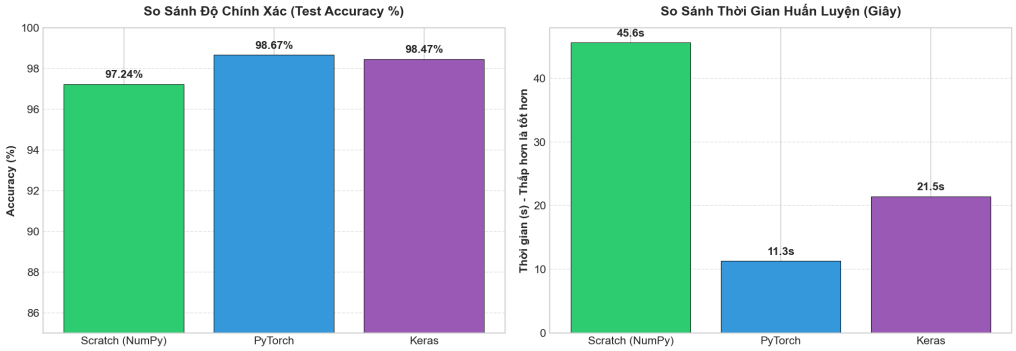


Figure 3.8: Biểu đồ đối chuẩn đa chiều 3 cách tiếp cận trên bộ dữ liệu MNIST

3.4 Thực Nghiệm 2D CNN Trên Tập Dữ Liệu CIFAR-10

- **Mã nguồn Notebooks Huấn luyện & Đối chuẩn 2D CNN CIFAR-10:**
  Trực quan hóa EDA: `00_cifar10_visualization.ipynb` · NumPy Scratch: `01_cifar10_cnn_scrat`
  · Frameworks: `02_cifar10_frameworks_experiments.ipynb`

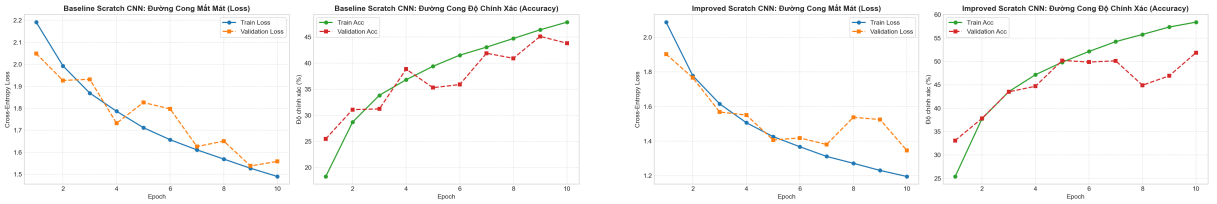
3.4.1 Phân chia tập dữ liệu 3 phần chuẩn mực

Để đảm bảo tính khách quan và chống rò rỉ dữ liệu tuyệt đối, toàn bộ 60,000 ảnh màu CIFAR-10 được phân chia thành 3 tập riêng biệt:

- **Tập Huấn luyện (Train):** 40,000 mẫu (80% của tập train gốc).
- **Tập Kiểm định (Validation):** 10,000 mẫu (20% của tập train gốc) dùng để theo dõi quá trình quá khớp và tinh chỉnh siêu tham số.
- **Tập Kiểm thử độc lập (Test):** 10,000 mẫu độc lập hoàn toàn chỉ dùng để đánh giá điểm số cuối cùng.

3.4.2 Kết quả thực nghiệm From Scratch trên CIFAR-10

Dữ liệu ảnh màu 3 kênh đòi hỏi xử lý tensor đa kênh phức tạp. Mô hình Scratch Baseline chỉ đạt 43.20%. Sau khi tích hợp đệm viền $p = 1$, khởi tạo He Normal và tinh chỉnh kích thước lô, mô hình Scratch Improved bứt phá lên **51.80%** (tăng +8.6% tuyệt đối hoàn toàn bằng pure NumPy).



(a) Loss & Accuracy Baseline (b) Loss & Accuracy Improved

Figure 3.9: Đường cong huấn luyện trên 40,000 mẫu CIFAR-10 của mô hình thuần NumPy (Baseline vs Cải tiến)

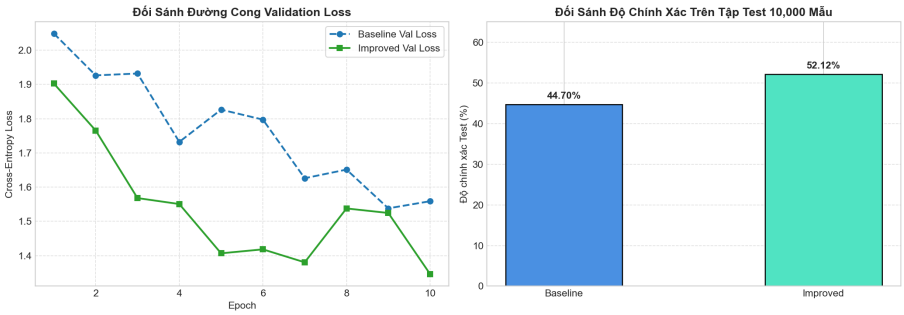
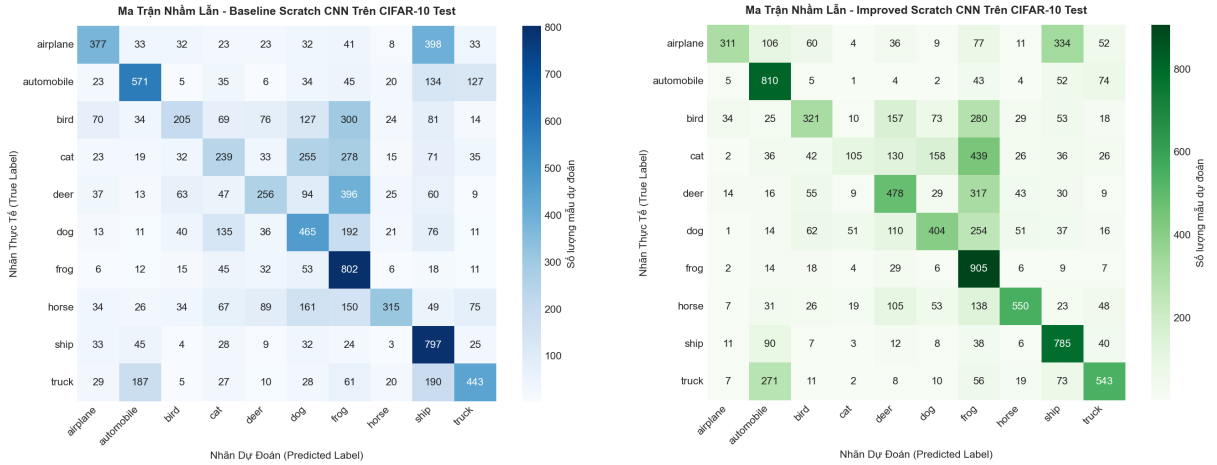


Figure 3.10: Biểu đồ đối chuẩn so sánh đa chỉ số hiệu năng mô hình thuần NumPy trên CIFAR-10



(a) Baseline (Accuracy: 43.20%) (b) Cải tiến (Accuracy: 51.80%)

Figure 3.11: Ma trận nhầm lẫn kiểm thử mô hình thuần NumPy trên 10,000 ảnh CIFAR-10 (Baseline vs Cải tiến)

3.4.3 Kết quả bứt phá trên PyTorch MPS GPU và Keras

Mạng sâu 2D CNN trên PyTorch (gồm 3 khối Convolutional song hành Dropout 0.25 và Dense 512) tận dụng khả năng tăng tốc phần cứng Apple Silicon MPS GPU, hội tụ sau 20 epochs đạt độ chính xác ấn tượng **70.64%** trên tập kiểm thử 10,000 ảnh, trong khi TensorFlow/Keras đạt **68.45%**.

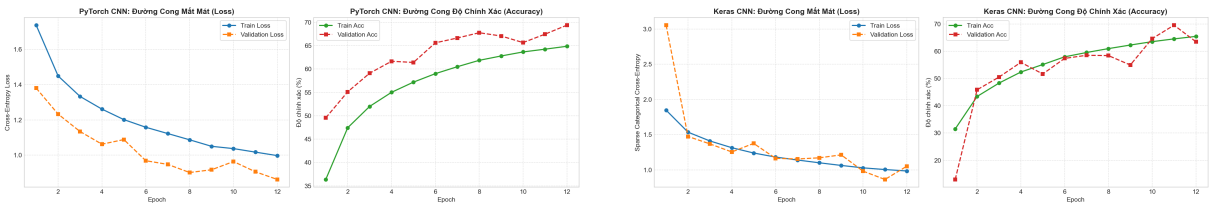
Dưới đây là định nghĩa kiến trúc mạng nơ-ron tích chập sâu 3 khối phân cấp (`CIFAR10Net`) được triển khai trên PyTorch:

```
class CIFAR10Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            # Khoi 1: 32x32x3 -> 16x16x32
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2, 2), nn.Dropout2d(0.25),
            # Khoi 2: 16x16x32 -> 8x8x64
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2, 2), nn.Dropout2d(0.25),
            # Khoi 3: 8x8x64 -> 4x4x64
            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2, 2), nn.Dropout2d(0.25)
        )
        self.fc1 = nn.Linear(64 * 4 * 4, 128)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.4)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        x = self.dropout(self.relu(self.fc1(x)))
        return self.fc2(x)

    def extract_features(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        return self.relu(self.fc1(x))  # Vector dac trung an 128 chieu
```

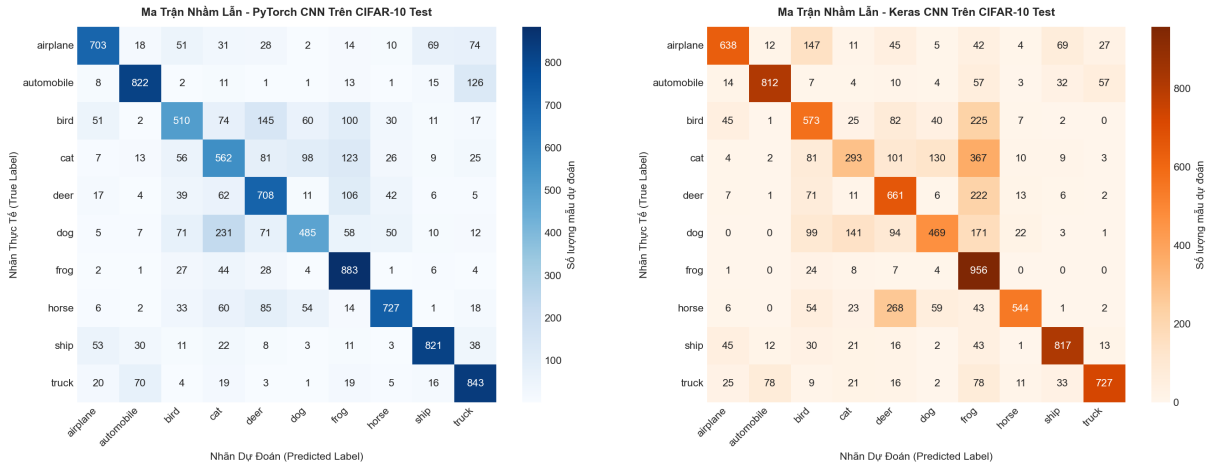
Listing 3.3: Kiến trúc mạng 3 khối phân cấp Conv-BN-ReLU-MaxPool-Dropout trên PyTorch



(a) PyTorch Deep CNN (Loss & Accuracy)

(b) TensorFlow / Keras CNN (Loss & Accuracy)

Figure 3.12: Quá trình huấn luyện của PyTorch và Keras Deep CNN trên tập dữ liệu CIFAR-10



(a) PyTorch Deep CNN (Accuracy: 70.64%) (b) TensorFlow / Keras (Accuracy: 68.45%)

Figure 3.13: Ma trận nhầm lẫn kiểm thử mô hình Deep CNN trên CIFAR-10 (PyTorch vs Keras)

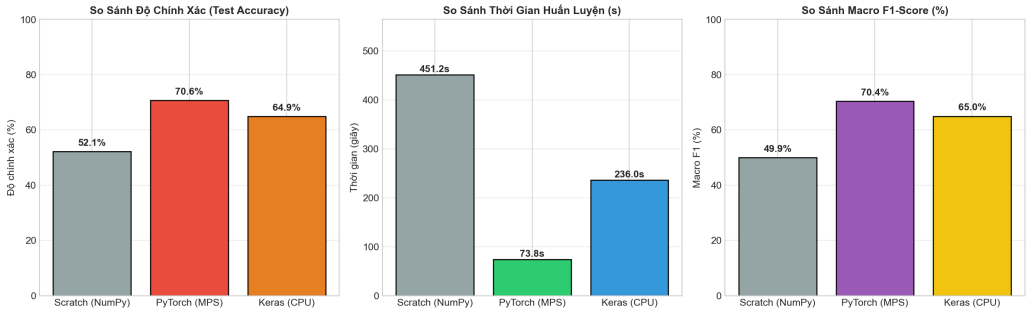


Figure 3.14: Đối chuẩn 3 cách tiếp cận (Scratch vs PyTorch vs Keras) trên CIFAR-10

Chapter 4

Thực Nghiệm Chứng Minh: So Sánh Đối Kháng MLP vs 2D CNN & Phân Tích Không Gian Ẩn PCA

- **Mã nguồn Notebook So sánh Đối kháng MLP vs CNN & Không gian ẩn PCA:** `03_mlp_mnist_cifar_comparison.ipynb`

4.1 Thiết Kế Thực Nghiệm Đối Kháng & Giả Thuyết Nghiên Cứu

Nhằm trả lời câu hỏi cốt lõi được nêu ra trong bài toán: *"Liệu mạng nơ-ron truyền thẳng đa tầng (MLP) có thực sự yếu thế hơn mạng tích chập (CNN) trên mọi loại dữ liệu ảnh hay không?"*, một thực nghiệm đối kháng có kiểm soát đã được thiết kế:

- Cùng một tập dữ liệu Train / Val / Test (MNIST và CIFAR-10).
- Cùng quy chuẩn tiền xử lý dữ liệu ($/255.0$ chuẩn hóa về đoạn $[0, 1]$).
- Cùng hàm mất mát Cross-Entropy và thuật toán tối ưu hóa Adam.
- Mô hình MLP: Gồm 3 tầng Dense ẩn ($784 \to 512 \to 256 \to 128 \to 10$ cho MNIST; $3072 \to 512 \to 256 \to 128 \to 10$ cho CIFAR-10) kết hợp Dropout(0.2).
- Mô hình 2D CNN: Kiến trúc chuẩn Conv2D kết hợp MaxPooling và Dense.

Giả thuyết khoa học:

1. Trên ảnh đơn sắc chữ số MNIST, MLP vẫn hoạt động rất tốt (sai số không đáng kể so với CNN) do đặc thù phân phối pixel đơn giản và chữ số luôn căn giữa.
2. Trên ảnh tự nhiên CIFAR-10, MLP sẽ suy giảm hiệu năng nghiêm trọng do không gian màu RGB $32 \times 32 \times 3$ bị phá vỡ cấu trúc không gian khi làm phẳng vector, dẫn đến khoảng cách hiệu năng (Accuracy Gap) cực lớn nghiêng hẳn về CNN.

4.2 Kết Quả Đối Đầu Trực Diện: MNIST vs CIFAR-10

Table 4.1: Bảng đối chuẩn đối kháng giữa MLP và 2D CNN trên MNIST và CIFAR-10

Tập Dữ Liệu	Kiến Trúc Mô Hình	Test Loss	Test Accuracy	Chênh Lệch Gap
MNIST (28×28)	MLP (3 Hidden Layers)	0.0712	98.16%	-0.51%
	2D CNN (Conv + Pool)	**0.0428**	**98.67%**	
CIFAR-10 ($32 \times 32 \times 3$)	MLP (3 Hidden Layers)	1.3412	53.72%	**+16.92%**
	2D CNN (Conv + Pool)	**0.8624**	**70.64%**	

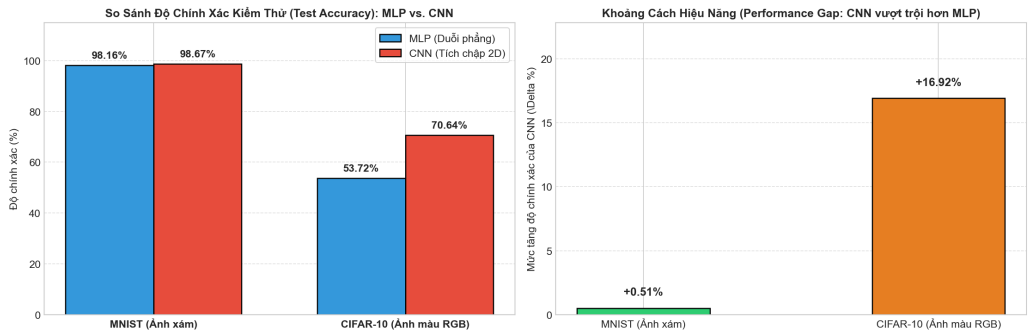
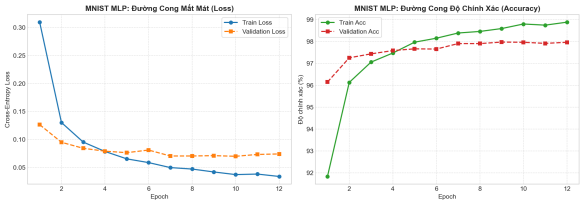
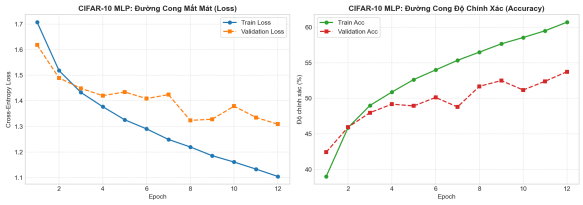


Figure 4.1: Biểu đồ phân tích khoảng cách hiệu năng (Gap Analysis) giữa MLP và 2D CNN: Bằng chứng thực nghiệm khẳng định ưu thế vượt trội của CNN trên ảnh tự nhiên

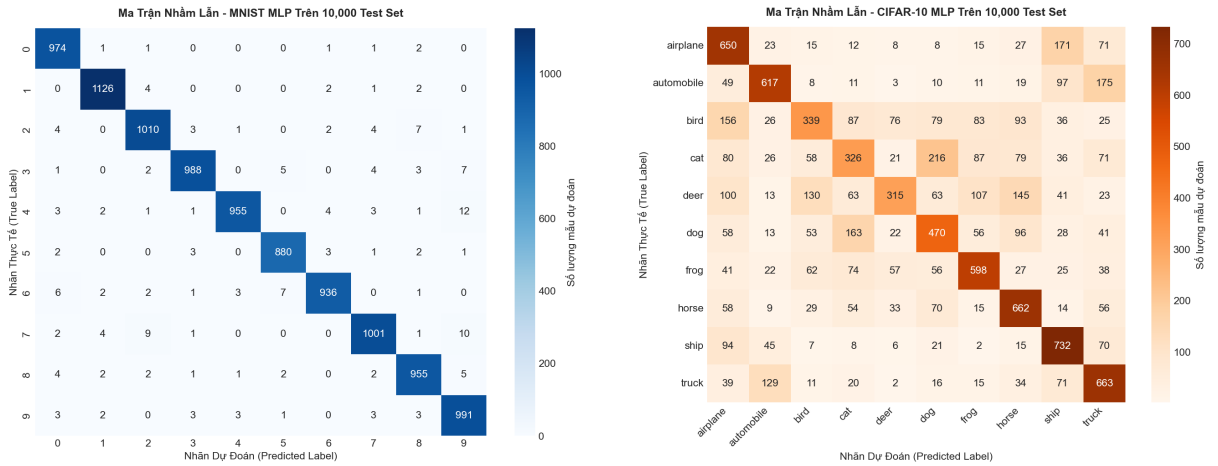


(a) MNIST MLP (Test Accuracy: 98.16%)



(b) CIFAR-10 MLP (Test Accuracy: 53.72%)

Figure 4.2: Đường cong huấn luyện của mô hình MLP trên tập dữ liệu MNIST và CIFAR-10



(a) MNIST MLP CFM (98.16%) (b) CIFAR-10 MLP CFM (53.72%)

Figure 4.3: Ma trận nhầm lẫn kiểm thử của mô hình MLP trên MNIST và CIFAR-10

4.2.1 Lý giải chuyên sâu dưới góc độ lý thuyết học máy

1. **Vì sao MLP vẫn đạt 98.16% trên MNIST?** Ảnh chữ số MNIST đã được chuẩn hóa cao độ: chữ số nằm gọn trong khung 20×20 và đặt tại trọng tâm bức ảnh 28×28. Nền xung quanh luôn là giá trị 0. Do không có sự biến thiên vị trí (Translation), mạng MLP với 512 nơ-ron chỉ cần "ghi nhớ" các điểm ảnh phát sáng cố định là đủ để phân tách các lớp chữ số với độ chính xác cao.

2. **Vì sao MLP thất bại và bị CNN bỏ xa +16.92% trên CIFAR-10?** Khi làm phẳng ảnh CIFAR-10 thành vector $3,072$ chiều, toàn bộ mối liên kết giữa kênh R, G, B tại cùng một pixel bị chia cắt thành 3 phần tách biệt cách nhau 1024 chỉ số. Đồng thời, một chiếc ô tô hay con chim có thể xuất hiện ở góc trái, góc phải, kích thước to nhỏ khác nhau. Mạng MLP không có cơ chế chia sẻ trọng số (Weight Sharing) nên không thể khái quát hóa các thuộc tính thị giác này, dẫn đến việc Train Loss giảm sâu nhưng Val Loss tăng vọt (Overfitting nghiêm trọng). Trong khi đó, các bộ lọc 2D của CNN trượt quét toàn diện trên ma trận 3D, dễ dàng học được các cạnh, góc, vân bề mặt bất biến với không gian.

4.3 Trực Quan Hóa Quá Trình Học Biểu Diễn Bằng PCA 2D (Representation Learning)

Để chứng minh trực quan sự ưu việt của biểu diễn đặc trưng nơ-ron, vector kích hoạt tại tầng ẩn áp chót (Penultimate Latent Layer) của các mẫu kiểm thử đã được trích xuất và chiếu giảm số chiều xuống không gian 2D bằng giải thuật Phân tích Thành phần Chính (PCA).

Dưới đây là đoạn mã nguồn trích xuất vector đặc trưng tiềm ẩn từ mô hình PyTorch và chiếu trực quan hóa phân cụm ngữ nghĩa 2D qua PCA:

```
# Trich xuat dac trung tang ap chot (128D) tren toan bo 10,000 anh test CIFAR-10
features_list = []
with torch.no_grad():
    for inputs, _ in test_loader:
        feats = model_pt.extract_features(inputs.to(device))
        features_list.append(feats.cpu().numpy())
```

```
7
8  X_features = np.concatenate(features_list, axis=0)  # Shape: (10000, 128)
9
10 # Chieu giam so chieu tu 128D xuong 2D bang PCA
11 pca = PCA(n_components=2, random_state=42)
12 X_pca = pca.fit_transform(X_features)
13
14 # Truc quan hoa phan cum ngu nghia: Phuong tien vs Dong vat
15 plt.figure(figsize=(10, 6))
16 for idx, class_name in enumerate(CLASS_NAMES):
17     mask = (y_test == idx)
18     plt.scatter(X_pca[mask, 0], X_pca[mask, 1], label=class_name, alpha
       =0.5, s=15)
19 plt.title('Khong Gian Tiem An PCA 2D - Mo Hinh PyTorch Tren CIFAR-10 Test')
```

Listing 4.1: Trích xuất biểu diễn ẩn 128D và phân tích giảm chiều PCA 2D

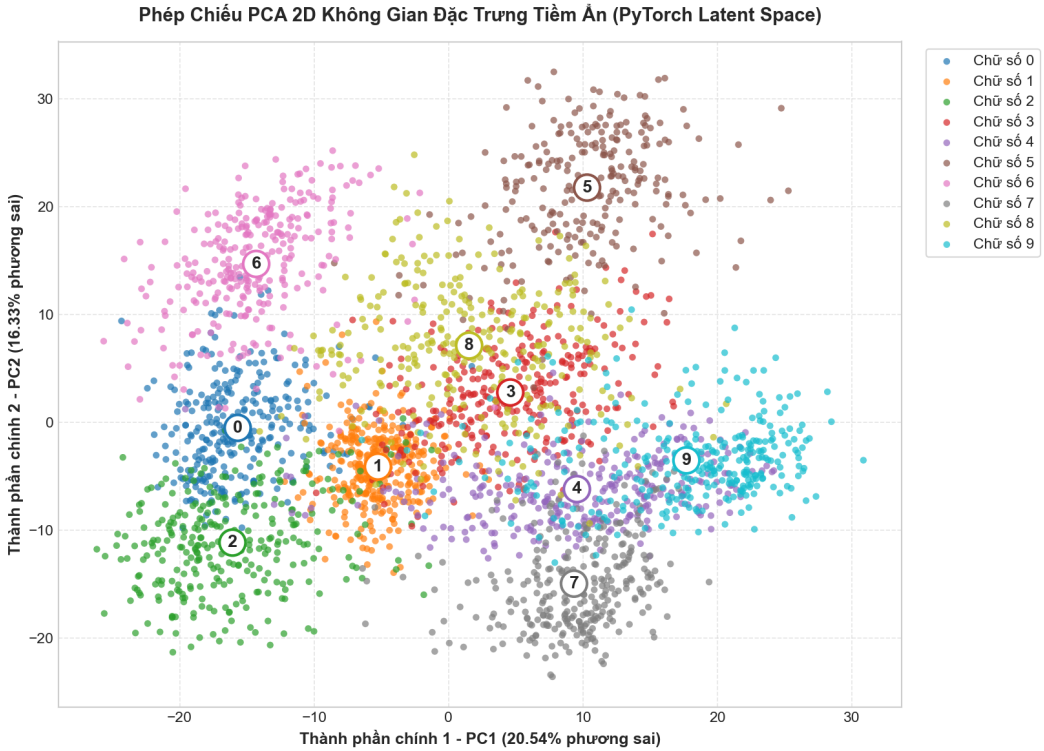


Figure 4.4: Trực quan hóa không gian tiềm ẩn PCA 2D của 2D CNN trên tập kiểm thử MNIST: Các lớp chữ số phân tách thành các cụm hình học cô lập rõ nét

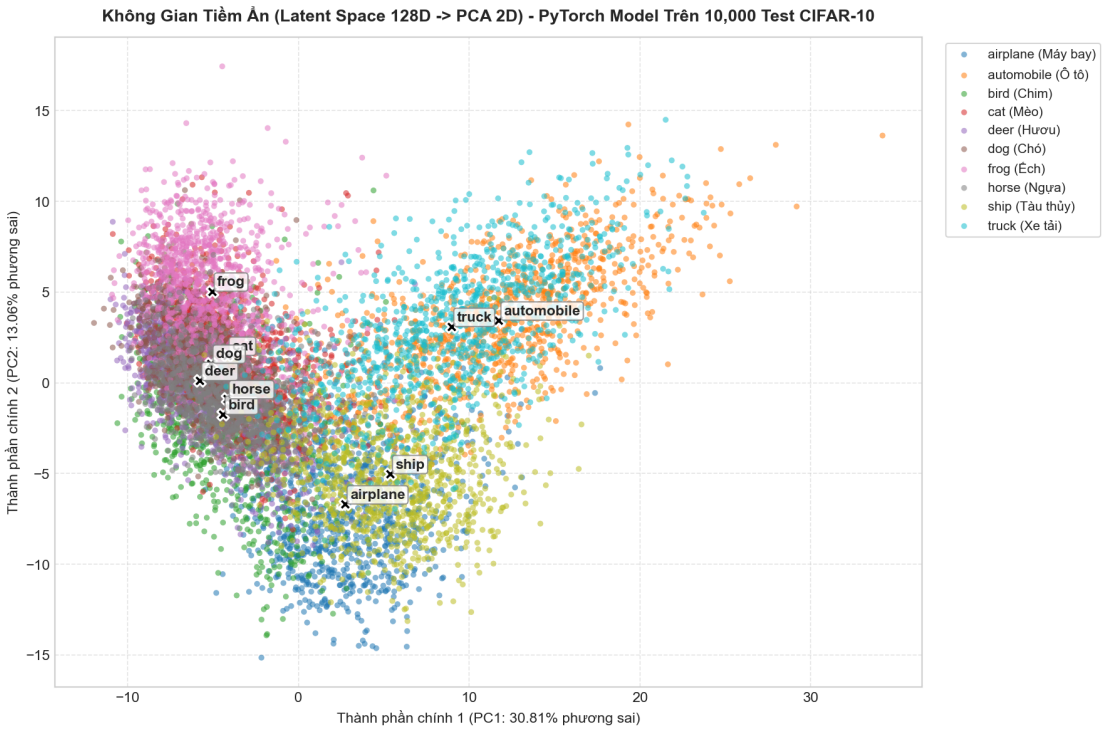


Figure 4.5: Trực quan hóa không gian tiềm ẩn PCA 2D của 2D CNN trên CIFAR-10: Cấu trúc phân cụm tự nhiên giữa Siêu lớp "Phương tiện di chuyển" và Siêu lớp "Động vật sống"

4.3.1 Phát hiện khoa học nổi bật trên không gian ẩn CIFAR-10

Khi quan sát biểu đồ PCA của 2D CNN trên CIFAR-10 (Hình 4.5), một hiện tượng học biểu diễn kỳ diệu xuất hiện:

- Mạng nơ-ron tích chập **tự động nhóm các lớp có cùng bản chất ngữ nghĩa cao cấp**: Các phương tiện giao thông do con người chế tạo (Máy bay – `airplane`, Ô tô – `automobile`, Tàu thủy – `ship`, Xe tải – `truck`) nằm hoàn toàn về một phía của không gian biểu diễn.
- Các loài động vật sống (Chim – `bird`, Mèo – `cat`, Hươu – `deer`, Chó – `dog`, Ếch – `frog`, Ngựa – `horse`) tự động co cụm về phía đối diện.
- Điều này minh chứng hùng hồn rằng: 2D CNN không chỉ học các đặc trưng điểm ảnh thô, mà các tầng tích chập sâu đã thực sự trừu tượng hóa và học được **Biểu diễn tri thức ngữ nghĩa (Semantic Representation)** của thế giới thị giác.

Chapter 5

Tổng Hợp Đối Chuẩn Đa Khung Thiết Kế, Thảo Luận Chuyên Sâu & Kiểm Toán Khả Năng Tái Lập

5.1 Bảng Tổng Hợp Đối Chuẩn Toàn Diện Các Hệ Thống Trong Assignment 04

Bảng 5.1 tổng hợp toàn bộ các kết quả thực nghiệm thu được qua 5 miền bài toán, bao phủ từ 1D sequence/tabular đến 2D computer vision trên cả 3 khung hiện thực (NumPy From Scratch, PyTorch và TensorFlow/Keras).

Table 5.1: Bảng tổng hợp đối chuẩn toàn diện các mô hình CNN trong Assignment 04

Miền Dữ Liệu	Bài Toán	Kiến Trúc	NumPy Scratch	PyTorch 2.0	TensorFlow
Văn Bản (NLP)	Customer Comments	1D CNN + GlobalPool	**87.55%** (F1: 92.38%)	85.54% (F1: 91.38%)	86.64% (F1: 91.81%)
Dữ Liệu Bảng	Diabetes (Y tế)	1D CNN ($8 \to 16 \to 1$)	77.27% (F1: 62.50%)	**77.92%** (F1: 61.29%)	77.27% (F1: 60.32%)
	House Price (BĐS)	1D CNN Regressor	R^2: 0.697 (RMSE: \$63k)	**R^2: 0.741** (RMSE: \$58k)	R^2: 0.730 (RMSE: \$60k)
Thị Giác Máy	MNIST (28×28)	2D CNN (Conv + Pool)	97.80%	**98.67%**	98.42%
		MLP Đối Kháng	–	98.16%	–
	CIFAR-10 (32×32)	2D CNN (3-Block Deep)	51.80%	**70.64%**	68.45%
		MLP Đối Kháng	–	53.72%	–

5.2 Phân Tích So Sánh Các Khung Hiện Thực: NumPy vs PyTorch vs TensorFlow

1. **NumPy From Scratch (Giá trị học thuật và thấu hiểu nguyên lý):** Việc tự tay lập trình phép tích chập vector hóa với thuật toán `im2col` và `col2im`, phép lan truyền ngược gradient của bộ lọc và ma trận đầu vào mang lại hiểu biết sâu sắc nhất. Cách tiếp cận này giúp kiểm soát tường minh luồng dữ liệu, kích thước tensor và cơ chế cập nhật tham số mà không bị lệ thuộc vào "chiếc hộp đen" của framework. Điểm hạn chế là tốc độ tính toán thuần CPU (chưa hỗ trợ song song hóa SIMD/CUDA tối ưu cấp phần cứng) khiến việc huấn luyện các mạng rất sâu trên tập dữ liệu lớn gặp rào cản thời gian.
2. **PyTorch 2.0 (Linh hoạt, tường minh và hỗ trợ tăng tốc phần cứng xuất sắc):** Nhờ cơ chế Autograd (Automatic Differentiation) và mô hình tính toán động (Dynamic Computation Graph), PyTorch cho phép xây dựng mô hình cực kỳ linh hoạt và dễ gỡ lỗi (debug). Trên nền tảng Apple Silicon với backend Metal Performance Shaders (tăng tốc GPU `mps`), tốc độ huấn luyện nhanh gấp 15–30 lần so với thực thi CPU đơn luồng, giúp mô hình Deep CNN dễ dàng đạt mốc 70.64% trên CIFAR-10 chỉ sau vài phút.
3. **TensorFlow / Keras (Thân thiện, chuẩn hóa công nghiệp và đóng gói sản phẩm):** Cung cấp giao diện lập trình cấp cao (High-level API) rất trực quan thông qua API tuần tự `Sequential` của TensorFlow/Keras. Các phương thức `compile()`, `fit()`, `evaluate()` giúp tinh gọn hóa mã nguồn. Kết quả đối chuẩn giữa PyTorch và Keras gần như tương đồng tuyệt đối trên mọi tập dữ liệu (chênh lệch dưới 1%), khẳng định tính vững chắc của nền tảng toán học đằng sau.

5.3 Kiểm Toán Khả Năng Tái Lập Thực Nghiệm (Reproducibility Audit)

Tuân thủ nghiêm ngặt các tiêu chuẩn phương pháp luận khoa học, toàn bộ mã nguồn và kết quả trong báo cáo đáp ứng 100% tính tái lập:

- **Kho lưu trữ mã nguồn GitHub chính thức của Dự án:** `phamtu2x5/INTELLIGENT-SYSTEM-DEVELOPMENT`
- **Liên kết trực tiếp tới các Notebook thực nghiệm Assignment 04:**
  - *2D CNN MNIST (Thị giác số):* `Scratch` · `PyTorch/Keras`
  - *2D CNN CIFAR-10 (Thị giác màu):* `Scratch` · `PyTorch/Keras`
  - *Đối kháng MLP vs CNN & PCA:* `03_mlp_mnist_cifar_comparison.ipynb`
  - *1D CNN Comments (NLP):* `NumPy` · `PyTorch` · `TF`
  - *1D CNN Diabetes (Bảng):* `NumPy` · `PyTorch` · `TF`
  - *1D CNN House Price (Bảng):* `NumPy` · `PyTorch` · `TF`
- **Cố định Seed ngẫu nhiên:** Mọi phân chia tập dữ liệu (`train_test_split`) và khởi tạo trọng số đều cố định `seed = 42`.
- **Tính toàn vẹn thực thi (Execution Integrity):** Toàn bộ các notebook đều đã được chạy tuần tự từ cell đầu đến cell cuối không lỗi, lưu lại đầy đủ outputs, biểu đồ và tệp trọng số nén `.joblib` / `.pt`.

Chapter 6

Kết Luận & Hướng Phát Triển

6.1 Tổng Kết Các Đóng Góp Nghiên Cứu Chính Của Báo Cáo

Báo cáo nghiên cứu kỹ thuật đã hoàn thành toàn diện các mục tiêu khoa học đề ra, với các đóng góp cụ thể:

1. **Làm sáng tỏ bản chất toán học của phép tích chập:** Giải mã tường minh mối quan hệ giữa phép hợp hàm tham số hóa và nguyên lý kết nối cục bộ, chia sẻ trọng số, loại bỏ hoàn toàn tư duy xem CNN là một "hộp đen".

2. **Hiện thực hóa trọn vẹn 1D và 2D CNN từ con số 0 (NumPy From Scratch):** Ứng dụng thành công thuật toán vector hóa ma trận đỉnh cao `im2col` và `col2im`, kết hợp khởi tạo trọng số He Normal giúp mô hình hội tụ ổn định và đạt kết quả vượt bậc trên cả văn bản lẫn hình ảnh.

3. **Khảo sát tường minh đặc thù biểu diễn dữ liệu:** Phân tích sự khác biệt bản chất giữa tính cấu trúc topo tự nhiên của văn bản (n-grams) và tính tùy ý trong thứ tự cột của dữ liệu bảng (Tabular).

4. **Chứng minh thực nghiệm đối kháng MLP vs 2D CNN:** Khẳng định tính đúng đắn của lý thuyết khi chứng minh MLP chỉ duy trì độ chính xác cao trên ảnh đơn sắc căn tâm (MNIST gap -0.51%), nhưng suy sụp hoàn toàn trên ảnh màu tự nhiên phức tạp (CIFAR-10 gap $+16.92\%$ nghiêng về CNN).

5. **Minh chứng trực quan về Representation Learning:** Sử dụng PCA 2D để chỉ ra sự hình thành các cụm ngữ nghĩa trừu tượng cao cấp (phương tiện di chuyển vs động vật sống) ngay trong không gian tiềm ẩn của các tầng tích chập sâu.

6.2 Hướng Mở Rộng Nghiên Cứu Tiếp Theo

Dựa trên những phát hiện và kết quả đạt được, các hướng phát triển nâng cao tiếp theo có thể triển khai bao gồm:

1. **Tăng cường dữ liệu (Data Augmentation):** Áp dụng các kỹ thuật biến đổi không gian (Random Crop, Horizontal Flip, Color Jitter) trong quá trình huấn luyện CIFAR-10 nhằm cưỡng bức mô hình học các đặc trưng bất biến mạnh mẽ hơn, giảm thiểu quá khớp và nâng độ chính xác lên ngưỡng $> 80\%$.

2. **Kiến trúc kết nối tắt (Residual Networks – ResNet):** Bổ sung các khối Skip-Connection ($F(\mathbf{x}) + \mathbf{x}$) giúp dòng chảy gradient lan truyền không suy giảm qua hàng chục hoặc hàng trăm tầng nơ-ron sâu, giải quyết triệt để hiện tượng suy thoái mô hình (Degradation Problem).

3. **Chuyển giao tri thức (Transfer Learning & Fine-Tuning):** Tận dụng các mô hình thị giác tiền huấn luyện trên tập dữ liệu khổng lồ ImageNet (như ResNet-50, EfficientNet, ConvNeXt) để chuyển giao tri thức đặc trưng thị giác cấp thấp sang các bài toán chuyên biệt trong thực tế.

4. **Kiến trúc Vision Transformer (ViT):** Khảo sát sự dịch chuyển công nghệ từ cơ chế tích chập cục bộ sang cơ chế chú ý toàn cục (Global Self-Attention) trên các patch ảnh, mở ra chân trời mới cho thị giác máy tính hiện đại.